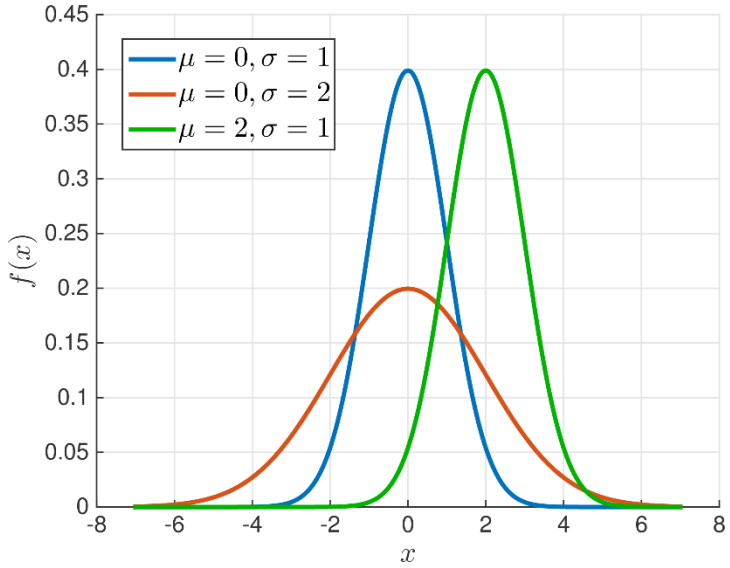
5. Generative models

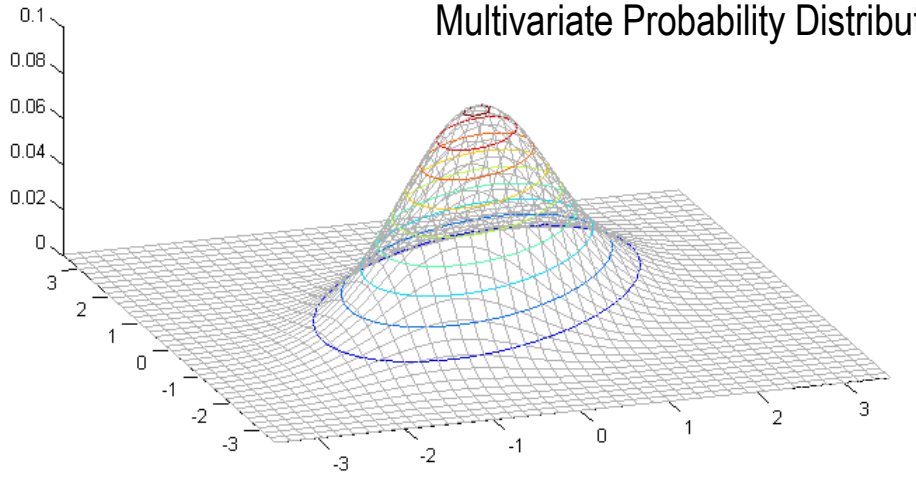
Continuous Probability Distribution

- **The probability of a continuous random variable**, which can take infinitely many possible values within a range.
  - PDF (Probability Density Function)
  - Defined over intervals
  - Total Area = 1
  - Single points have zero probabilities



1-dimensional space

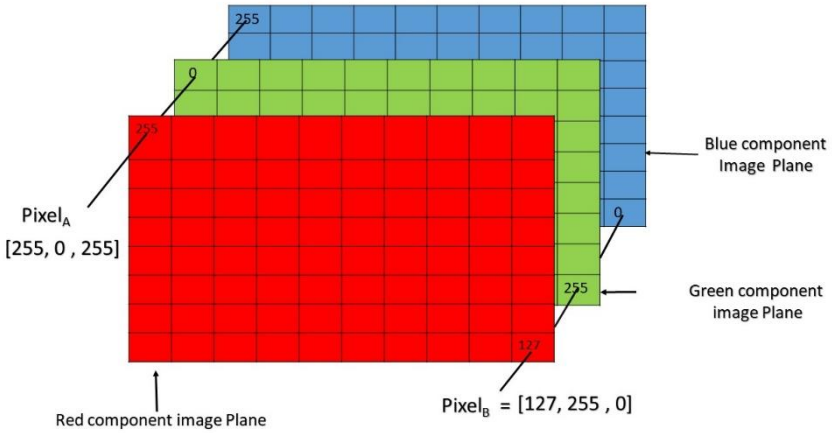
Multivariate Probability Distribution



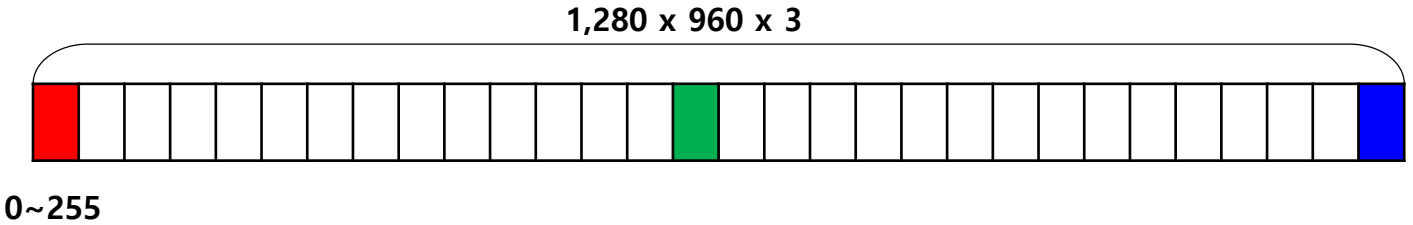
High-dimensional space

Pixel space

- **The total number of possible color combinations** for a 1280x960 RGB where each pixel have 256 values (0~255) per color channel
  - Enormous combinatorial space of digital images



Pixel of an RGB image are formed from the corresponding pixel of the three component images

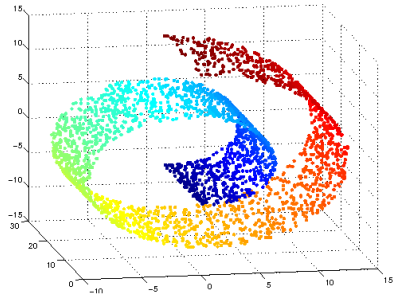


$$6.43 * 10^{8,877,735}$$

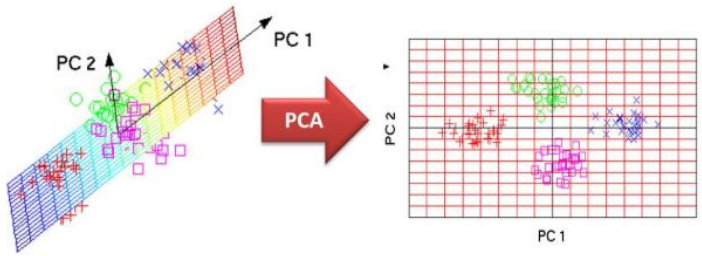
https://www.youtube.com/watch?v=EhndHhIvWWw&list=PLmkOM-kwIYM7p6Sq3iK8-f7VybXXbpOhu&index=2

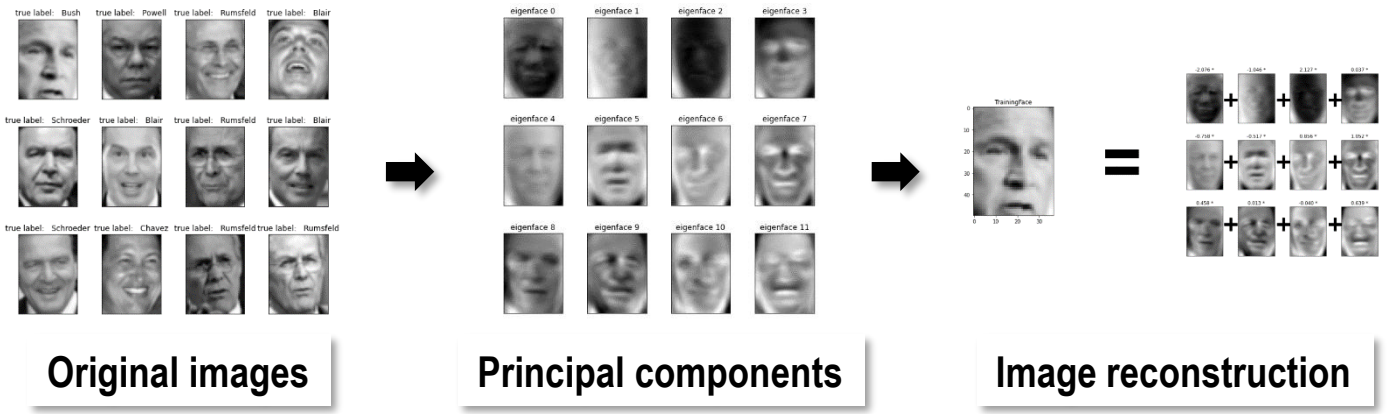
Manifold / PCA / Latent space (1/2)

- **Manifold**
  - A low-dimensional structure embedded in high-dimensional space



- **PCA (Principal Component Analysis)**
  - Linear dimensionality reduction technique that projects high-dimensional data onto a lower-dimensional space preserving as much variance (information) as possible
  - Each data point can be represented as a lower-dimensional vector (i.e., latent vector)

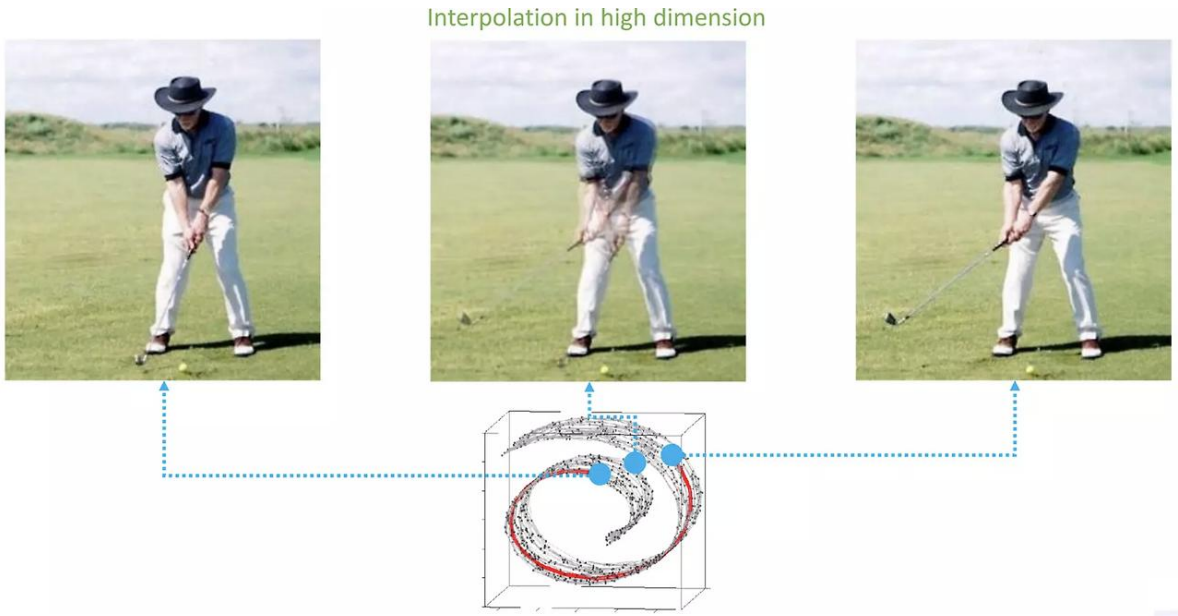




- **Latent space**
  - An abstract vector space that summarizes data with compact and meaningful representations
  - In deep learning, the latent space is a compressed vector space where data is represented by its essential underlying features

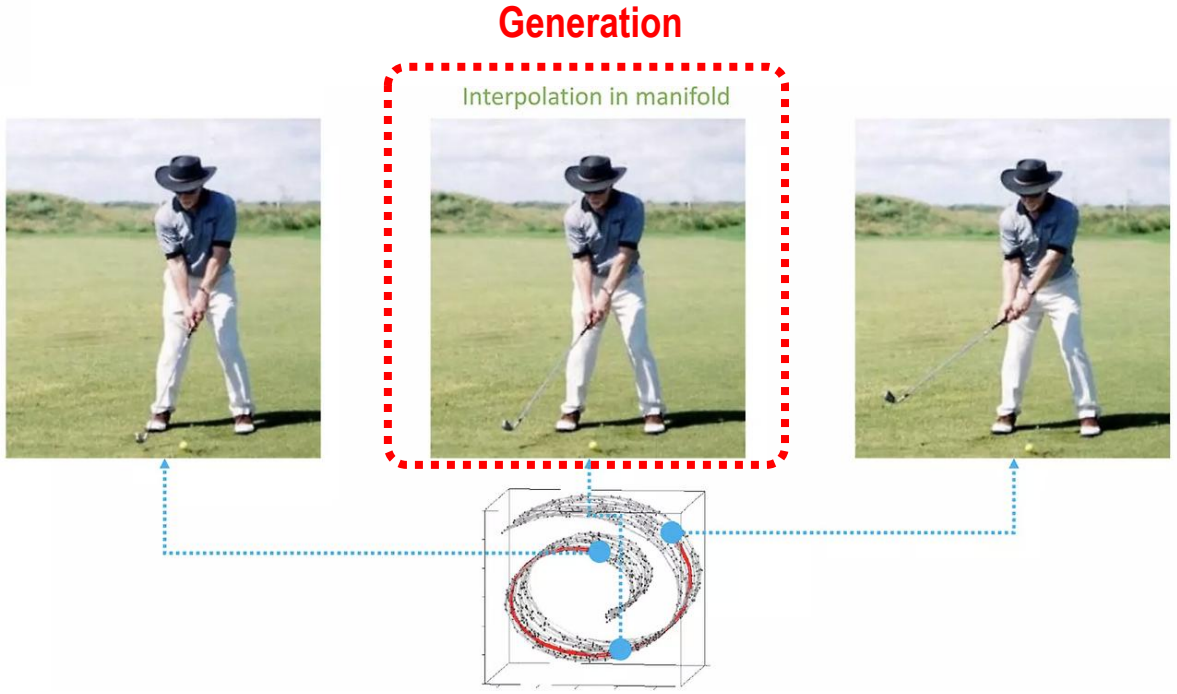
https://ashutoshtripathi.com/2019/07/11/a-complete-guide-to-principal-component-analysis-pca-in-machine-learning/
https://www.geeksforgeeks.org/machine-learning/ml-face-recognition-using-eigenfaces-pca-algorithm/
https://greatjoy.tistory.com/51

Manifold / PCA / Latent space (2/2)



Interpolation in high dimensional space

Interpolation in the latent space
→ Reconstruction (i.e., decoding)



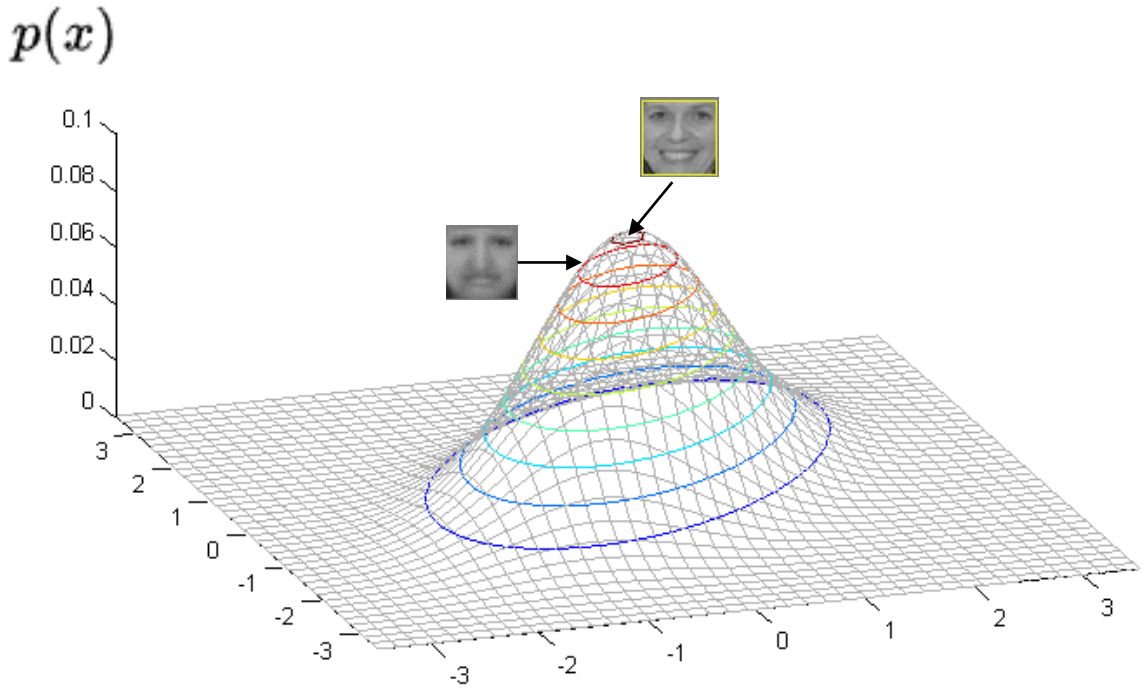
https://bkshin.tistory.com/entry/%EC%BB%B4%ED%93%A8%ED%84%B0-%EB%B9%84%EC%A0%84-7-%EC%98%A4%ED%86%A0%EC%9D%B8%EC%BD%94%EB%8D%94AutoEncoder%EC%99%80-%EB%A7%A4%EB%8B%88%ED%8F%B4%EB%93%9C-%ED%95%99%EC%8A%B5Manifold-Learning

Generative model

- Generative model is a type of machine learning model
  - learns the underlying distribution of data and
  - can generate new, synthetic samples that resemble the original data

- So, if the underlying distribution is learned successfully,
  - it can generate new data that have average statistical characteristics

Generative model =
Learning distribution of dataset
→ Sampling !



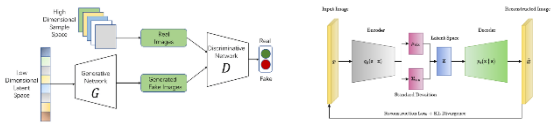
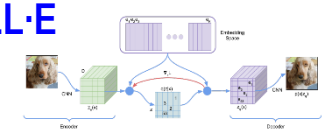
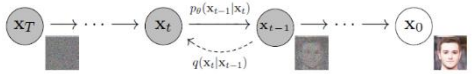
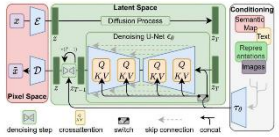


Generated new data

Original data

Overview

- Technical trend of '**Image Generative Models**'

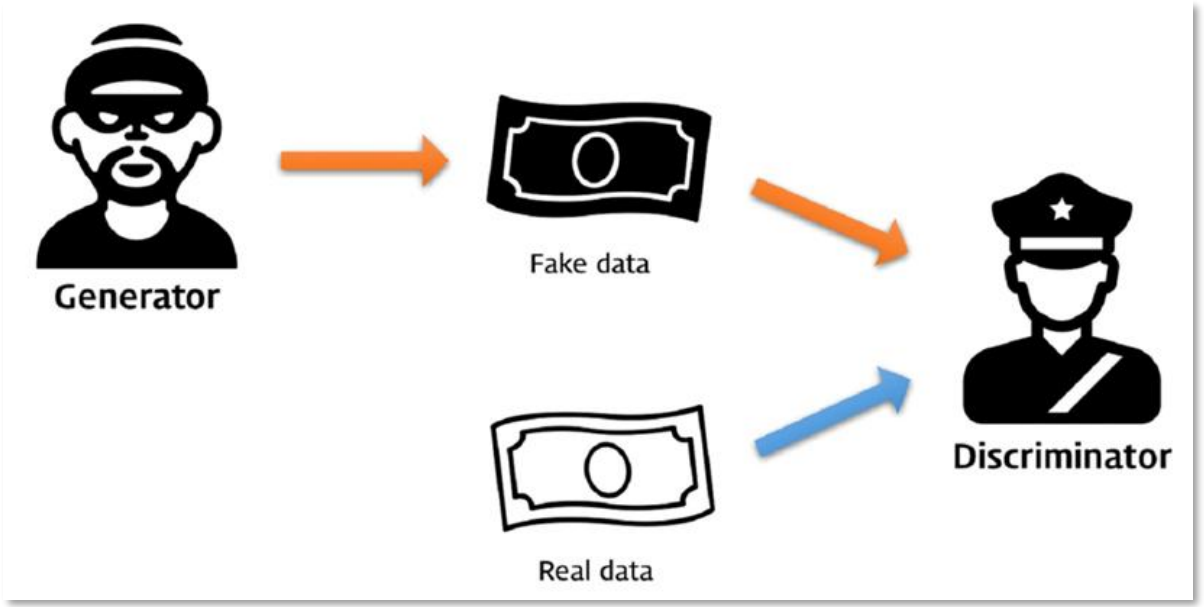
Generation	Representative Models	Technology Turning Point	Summary
1st Generation (2014–2017)	**GAN, VAE** 	Early deep generative models	Established fundamental neural generative frameworks
2nd Generation (2018–2021)	StyleGAN, BigGAN, **VQ-VAE, DALL·E** 	Latent space & scalability advancements	Improved fidelity and semantic control via enhanced latent representations
3rd Generation (2020–2022)	**DDPM**, DDIM, Guided Diffusion 	Likelihood-based diffusion modeling	Achieved robust training and superior sample quality through iterative denoising
4th Generation (2022–2023)	**Stable Diffusion**, ControlNet 	Latent diffusion & structural conditioning	Enabled fine-grained controllability and efficient text-to-image generation Stable_diffusion_online
5th Generation (2024–)	DiT	Real-time & transformer-based diffusion	Moved toward fast, scalable, and unified generative architectures

GAN
(Generative Adversarial Network)

[GAN]

- **Motivation**
  - Difficulties in training deep generative models
    - Markov chains → Computationally expensive and slow to converge
    - Variational inference → Require an explicit encoder or complex posterior approximations

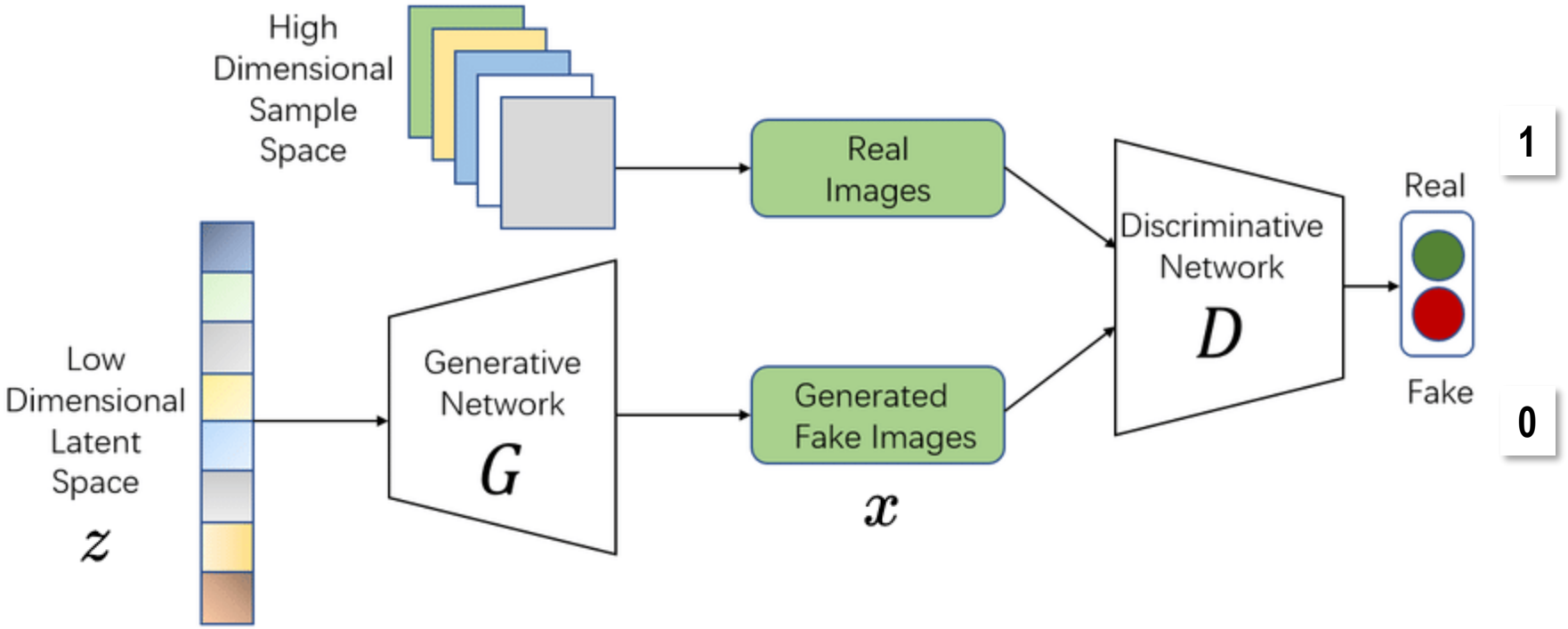
- **Goal**
  - Establishing a novel framework for estimating generative models via an adversarial process
    - Generative model (G) → Captures the data distribution
    - Discriminative model (D) → Estimates the probability that a sample came from the training data rather than G
  - Minimax two-player game, where G eventually produces samples indistinguishable from real data



https://bluediary8.tistory.com/146

Overview (1/2)

- **Overall architecture**



https://www.linkedin.com/pulse/what-generative-adversarial-networks-gans-sushant-babbar-qpc9c

Overview (2/2)

- **Key concept**

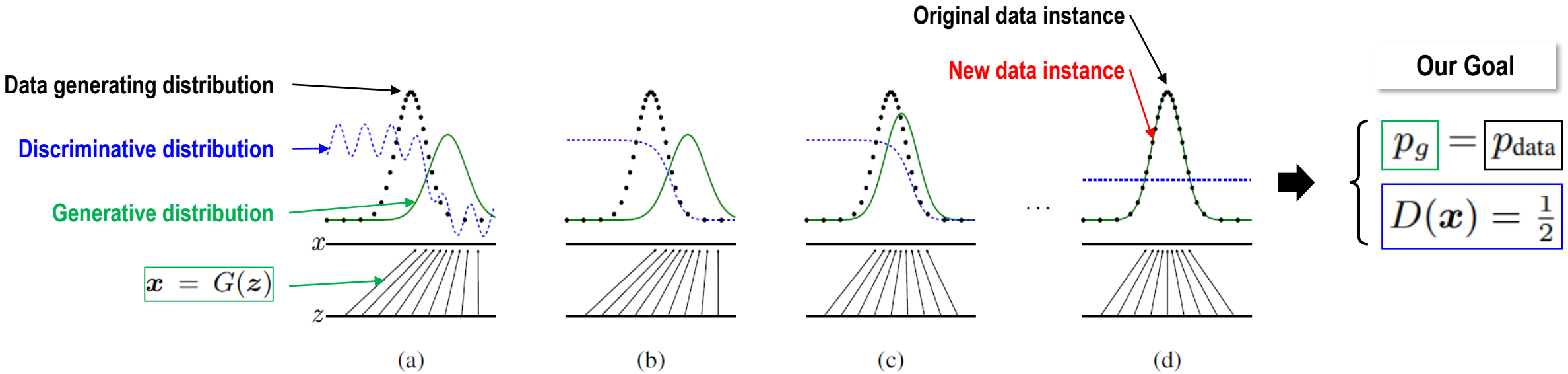
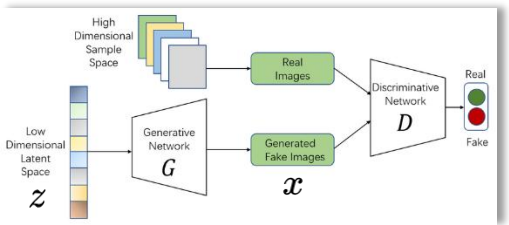


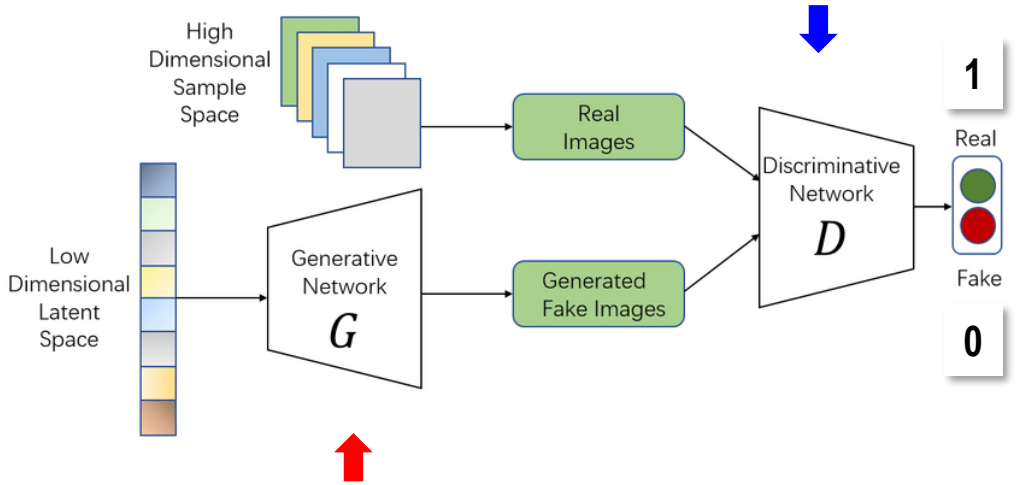
Figure 1: Generative adversarial nets are trained by simultaneously updating the **d**iscriminative distribution (D, blue, dashed line) so that it discriminates between samples from the data generating distribution (black, dotted line) $p_{\boldsymbol{x}}$ from those of the **g**enerative distribution p_g (G) (green, solid line). The lower horizontal line is the domain from which $\boldsymbol{z}$ is sampled, in this case uniformly. The horizontal line above is part of the domain of $\boldsymbol{x}$. The upward arrows show how the mapping $\boldsymbol{x} = G(\boldsymbol{z})$ imposes the non-uniform distribution p_g on transformed samples. G contracts in regions of high density and expands in regions of low density of p_g. (a) Consider an adversarial pair near convergence: p_g is similar to p_{data} and D is a partially accurate classifier. (b) In the inner loop of the algorithm D is trained to discriminate samples from data, converging to $D^*(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$. (c) After an update to G, gradient of D has guided $G(\boldsymbol{z})$ to flow to regions that are more likely to be classified as data. (d) After several steps of training, if G and D have enough capacity, they will reach a point at which both cannot improve because $p_g = p_{\text{data}}$. The discriminator is unable to differentiate between the two distributions, i.e. $D(\boldsymbol{x}) = \frac{1}{2}$.



Training (1/2)

- **Generate a random noise vector** and pass it through the **generator to produce a sample**
- **Pass the sample through the discriminator** to get a probability value
- **Compute the loss of the generator and the discriminator** by comparing the probability value with the desired output
- **Update the generator and discriminator parameters**

$$+ \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right]$$



$$- \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)$$

Loss

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

***x* : real**
→ *D*(*x*) = 1
→ log *D*(*x*) : maximize

***G*(*z*) : fake**
→ *D*(*G*(*z*)) = 0 (fake)
→ log (1 - *D*(*G*(*z*)) : maximize

***G*(*z*) : fake**
→ *D*(*G*(*z*)) = 1 (real)
→ log (1 - *D*(*G*(*z*)) : minimize

https://www.linkedin.com/pulse/what-generative-adversarial-networks-gans-sushant-babbar-qpc9c

Training (2/2)

- **Pseudo code**

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Global optimality (1/2)

- **Proposition**: For G fixed, the optimal discriminator D is

$$V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

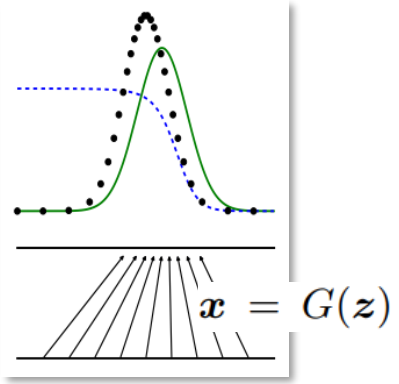
$$\mathbb{E}[X] = \int_x x f(x)\, dx$$

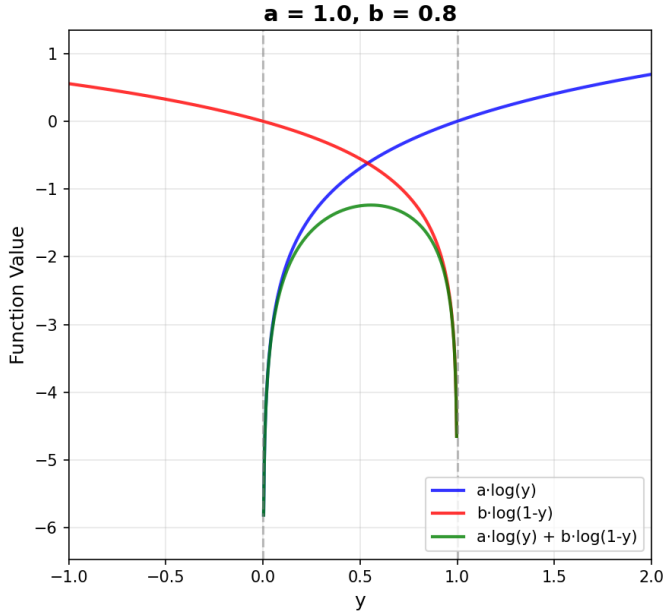
$$\begin{aligned} V(G,D) &= \int_{\boldsymbol{x}} p_{\text{data}}(\boldsymbol{x}) \log(D(\boldsymbol{x}))dx + \int_z p_{\boldsymbol{z}}(\boldsymbol{z}) \log(1 - D(g(\boldsymbol{z})))dz \\ &= \int_{\boldsymbol{x}} p_{\text{data}}(\boldsymbol{x}) \log(D(\boldsymbol{x})) + p_g(\boldsymbol{x}) \log(1 - D(\boldsymbol{x}))dx \end{aligned}$$

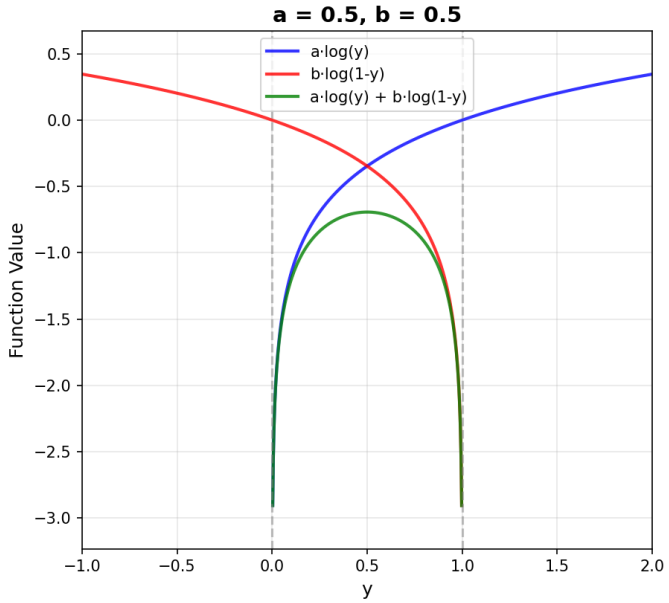
$$y \to a\log(y) + b\log(1-y) \quad \text{maximum in } [0,1] \text{ at } \tfrac{a}{a+b}$$

$$D_G^*(\boldsymbol{x}) = \frac{p_{data}(\boldsymbol{x})}{p_{data}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$$

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$







Global optimality (2/2)

- **Theorem**: Global minimum of the virtual training criterion is achieved when $p_g = p_{\text{data}}$

$$
\begin{aligned}
C(G) &= \max_D V(G, D) \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}}[\log D_G^*(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p_{\boldsymbol{z}}}[\log(1 - D_G^*(G(\boldsymbol{z})))] \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}}[\log D_G^*(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{x} \sim p_g}[\log(1 - D_G^*(\boldsymbol{x}))] \\
&= \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}}\left[\log \frac{p_{\text{data}}(\boldsymbol{x})}{P_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x} \sim p_g}\left[\log \frac{p_g(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}\right] \\
&= -\log(4) + KL\left(p_{\text{data}} \,\middle\|\, \frac{p_{\text{data}} + p_g}{2}\right) + KL\left(p_g \,\middle\|\, \frac{p_{\text{data}} + p_g}{2}\right) \\
&= 2 * \text{JSD}\left(p_{data} \,||\, p_g\right) - \log(4)
\end{aligned}
$$

For G fixed, the optimal discriminator D is

$$D_G^*(\boldsymbol{x}) = \frac{p_{data}(\boldsymbol{x})}{p_{data}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$$

It become 0.5

$$\text{JSD}\left(p||q\right) = \frac{1}{2}\text{KL}\left(p||\frac{p+q}{2}\right) + \frac{1}{2}\text{KL}\left(q||\frac{p+q}{2}\right)$$

It become zero when $p_g = p_{\text{data}}$

$$\min_G \max_D V(D, G) = \mathbb{E}_{\boldsymbol{x} \sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z} \sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Experiments

- **Visualization of samples from the model (i.e., from z)**
  - Not cherry-picked, Not memorized the training set

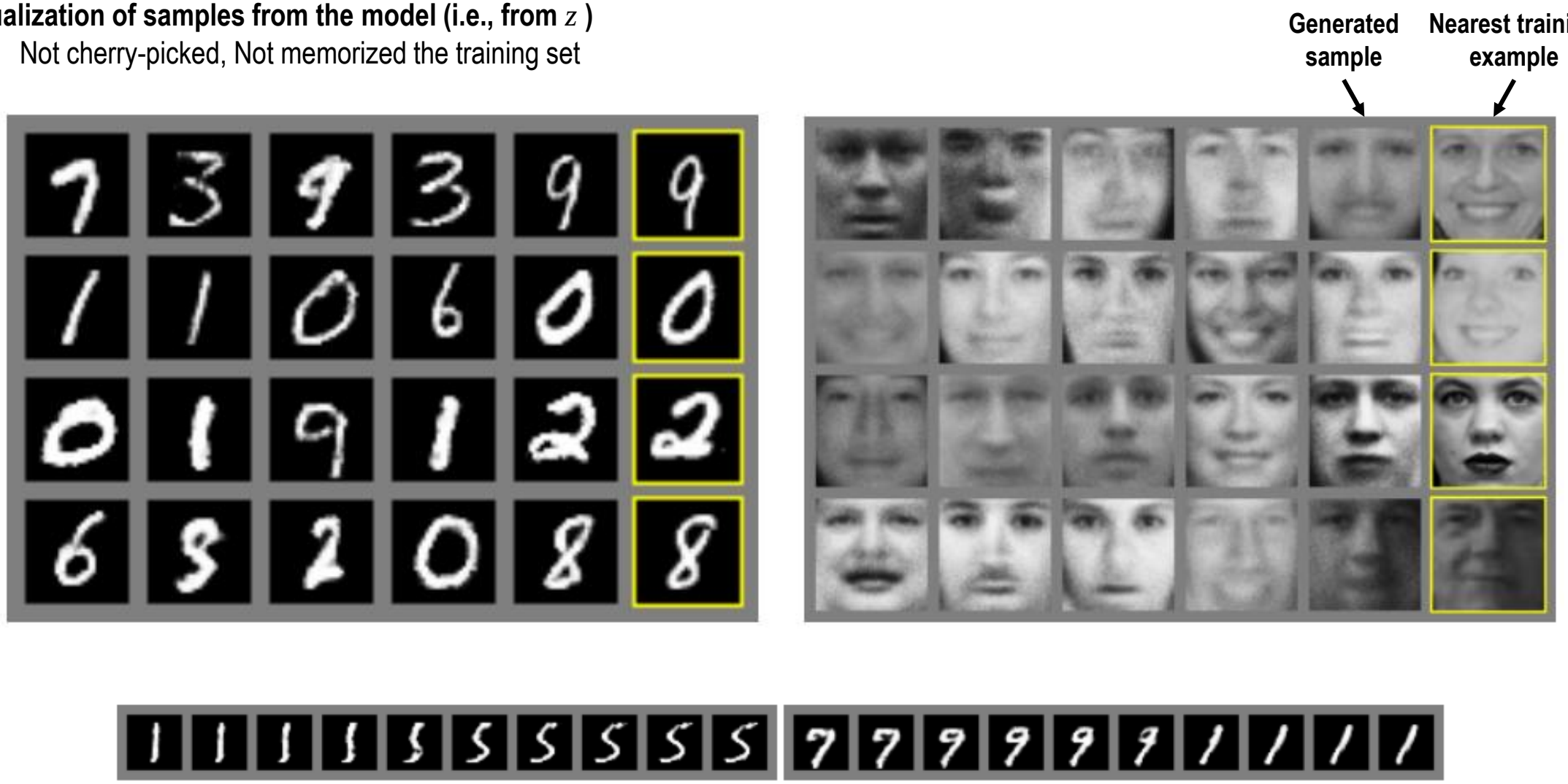


Figure 3: Digits obtained by linearly interpolating between coordinates in z space of the full model.

Summary

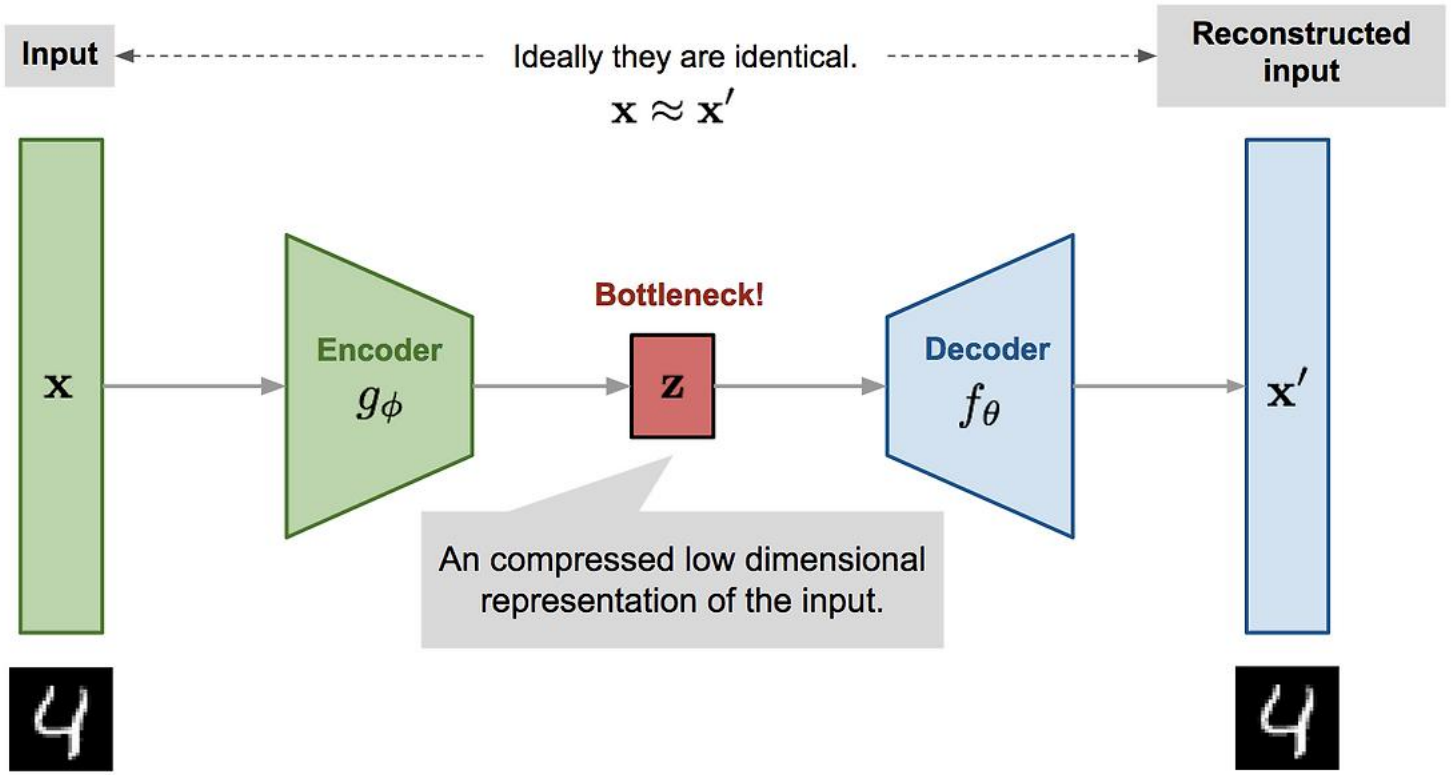
- **Limitations**
  - Training instability
    - Hard to balance the Generator and Discriminator
  - Mode collapse
    - Failing to capture the full diversity of the data
  - Vanishing gradient
    - If the Discriminator is too perfect $\rightarrow$ Generator can not get useful feedback to improve

Latent Representation

(Recap.) Auto-Encoder

- **Motivation**
  - Dimensionality reduction → Lower dimensional representation
  - Data compression → Removing noise and redundant features

- **Goal**
  - Unsupervised feature learning → Finding the Manifold (i.e., underlying structure)

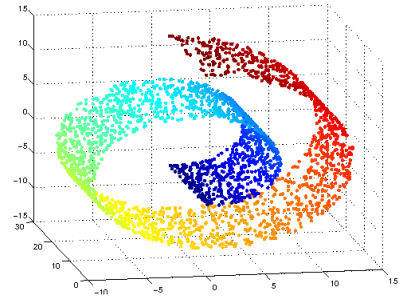


https://bkshin.tistory.com/entry/%EC%BB%B4%ED%93%A8%ED%84%B0-%EB%B9%84%EC%A0%84-7-%EC%98%A4%ED%86%A0%EC%9D%B8%EC%BD%94%EB%8D%94AutoEncoder%EC%99%80-%EB%A7%A4%EB%8B%88%ED%8F%B4%EB%93%9C-%ED%95%99%EC%8A%B5Manifold-Learning

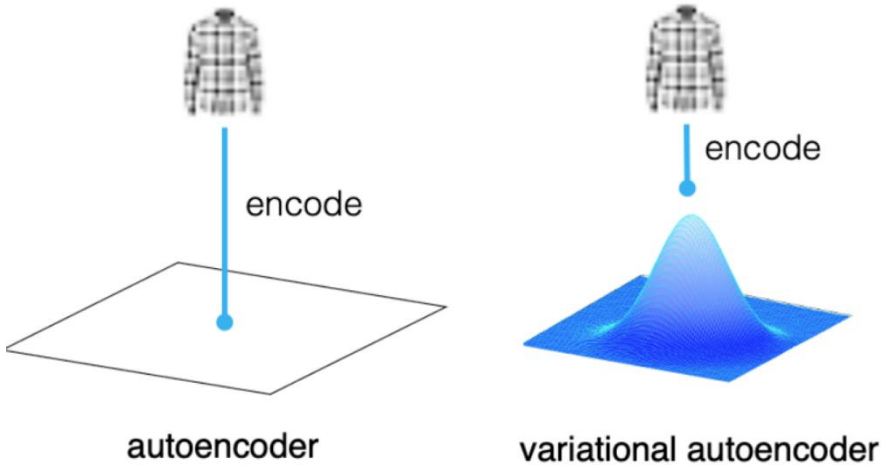
[Variational Auto-Encoder]

Auto-Encoding Variational Bayes, ICLR 2014

- **What does the 'Variational' means?**
  - It came from 'Variational inference' in statistics.
    → A method that approximates complex probability distributions by transforming it into an optimization problem



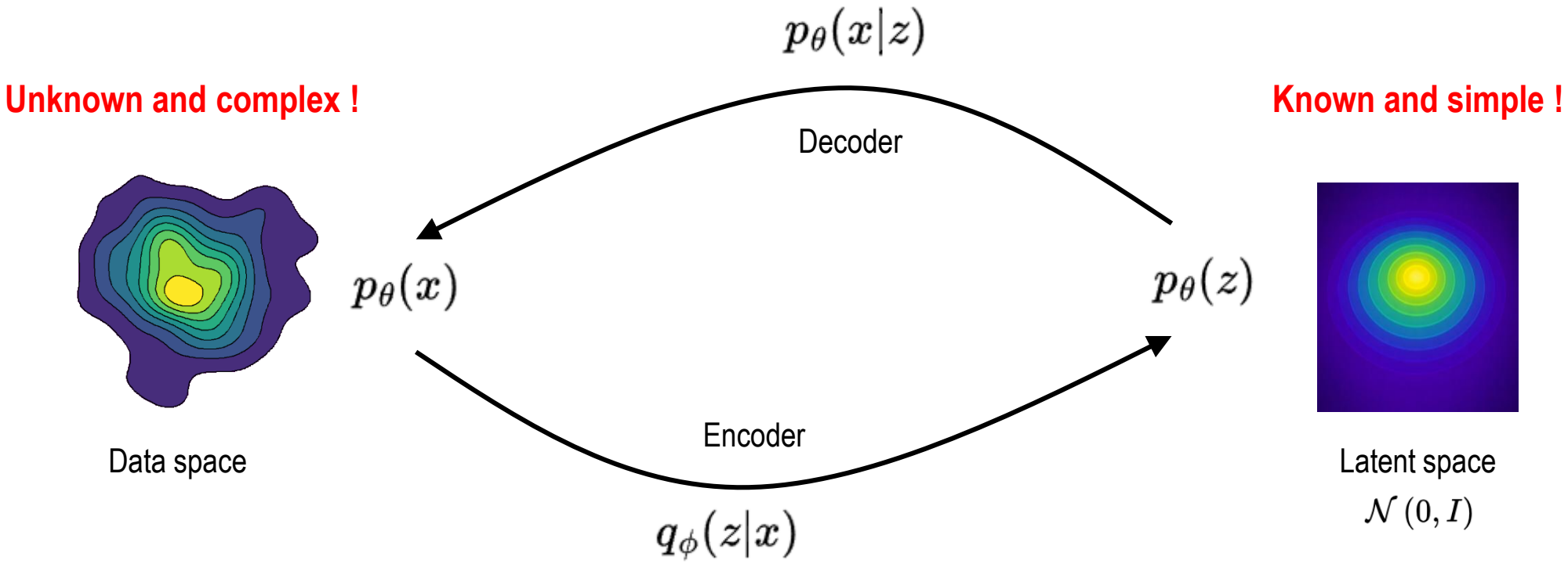
- **Key meanings of 'Variational' in VAE**
  - From points to distributions
    - Mapping data to a probability distribution (i.e., mean and variance)
  - Approximation of the intractable distribution
    - Intractable posterior distribution $p(z|x)$ with a simple one $q(z|x)$, e.g., Gaussian
  - Continuous latent space
    - Ensuring the latent space is regularized and continuous → Smooth interpolation and meaningful data generation from any sampled point



https://vitalflux.com/autoencoder-vs-variational-autoencoder-vae-difference/

Overview (1/2)

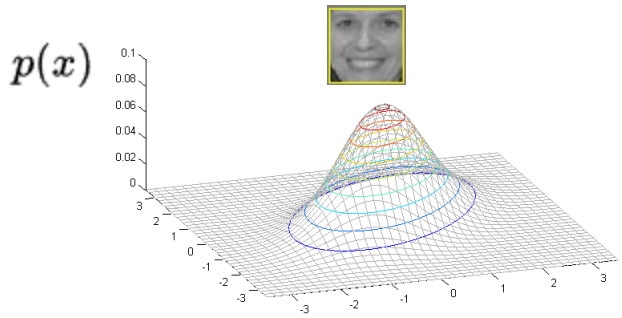
- **Key idea**



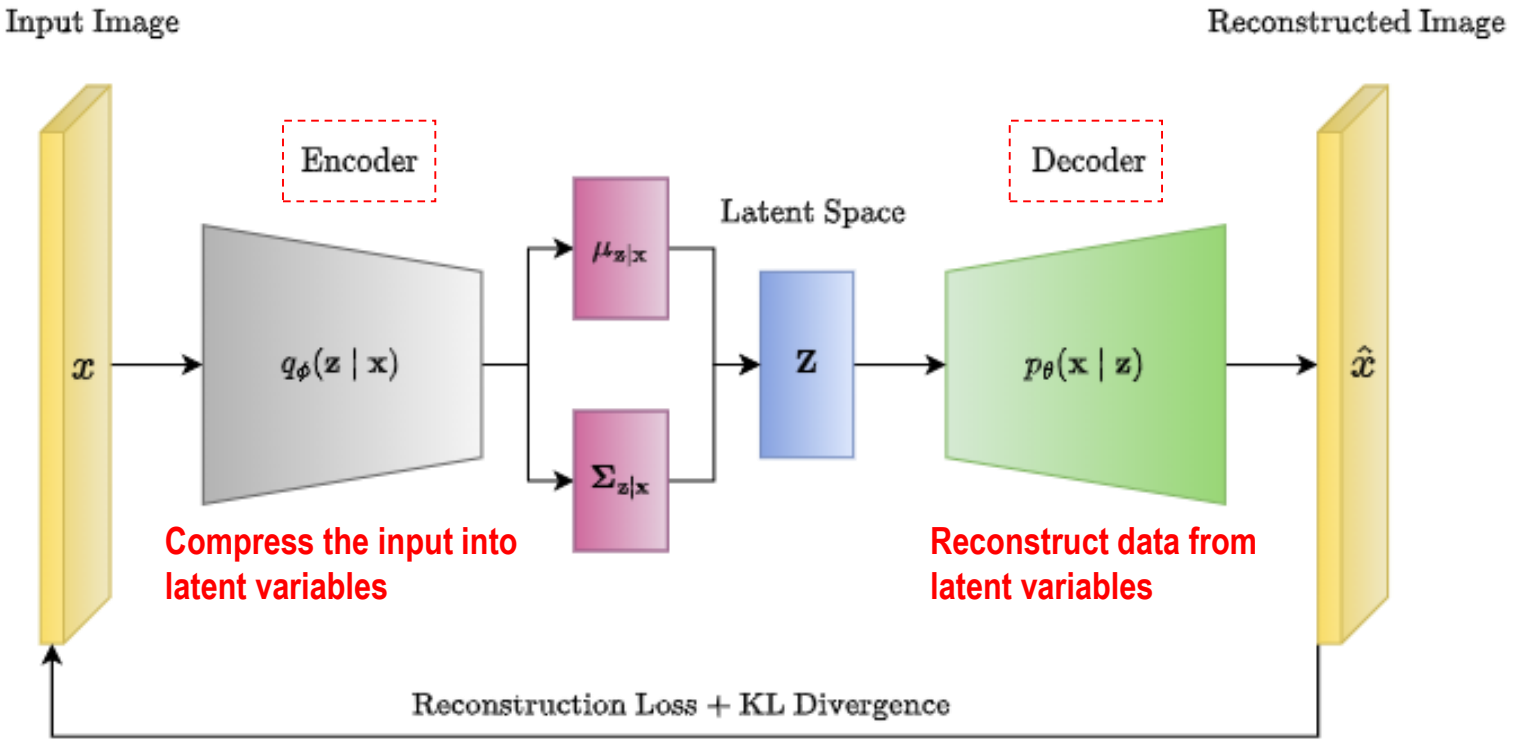
Overview (2/2)

- **Modeling the probability of image** based on the maximum likelihood, but it is intractable.

$$\arg\max_{\theta} \left[p_\theta(x) = \int_z p_\theta(x,z) = \int_z p_\theta(x|z) p_\theta(z) \right]$$



- Thus, VAE solves this by introducing
  - Encoder to approximate the posterior $q_\phi(z|x)$
  - Decoder to model $p_\theta(x|z)$



https://sushant-kumar.com/blog/vae-variational-autoencoder

Loss function (1/3)

$$\int_z q(z|x) = 1 \qquad p(x) = \frac{p(z,x)}{p(z|x)} \qquad \mathbb{E}[X] = \int x f(x) \qquad D_{KL}(P||Q) = \int p(x) \log \frac{p(x)}{q(x)}$$

- **Method 1**

$$\begin{aligned}
\log(p_\theta(x)) &= \int_z q_\phi(z|x) \log(p_\theta(x)) \\
&= \int_z q_\phi(z|x) \log \frac{p_\theta(z,x)}{p_\theta(z|x)} \\
&= \int_z q_\phi(z|x) \log \left(\frac{p_\theta(z,x)}{q_\phi(z|x)} \frac{q_\phi(z|x)}{p_\theta(z|x)} \right) \\
&= \int_z q_\phi(z|x) \log \frac{p_\theta(z,x)}{q_\phi(z|x)} + \int_z q_\phi(z|x) \log \frac{q_\phi(z|x)}{p_\theta(z|x)} \\
&= \mathcal{L}(\theta, \phi; x) + D_{KL}(q_\phi(z|x)||p_\theta(z|x)) \\
&\geq \mathcal{L}(\theta, \phi; x)
\end{aligned}$$

Lower bound

$$\begin{aligned}
\mathcal{L}(\theta, \phi; x) &= \int_z q_\phi(z|x) \log \frac{p_\theta(z,x)}{q_\phi(z|x)} \\
&= \int_z q_\phi(z|x) \log \frac{p_\theta(z) p_\theta(x|z)}{q_\phi(z|x)} \\
&= -\int_z q_\phi(z|x) \log \frac{q_\phi(z|x)}{p_\theta(z)} + \int_z q_\phi(z|x) \log p_\theta(x|z) \\
&= -D_{KL}\left(q_\phi(z|x)||p_\theta(z)\right) + \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x|z)\right]
\end{aligned}$$

Regularization — Reconstruction accuracy

Analytically computable

$$-\frac{1}{2} \sum_{j=1}^{J} \left(1 + \log\left(\sigma_j^2\right) - \mu_j^2 - \sigma_j^2\right)$$

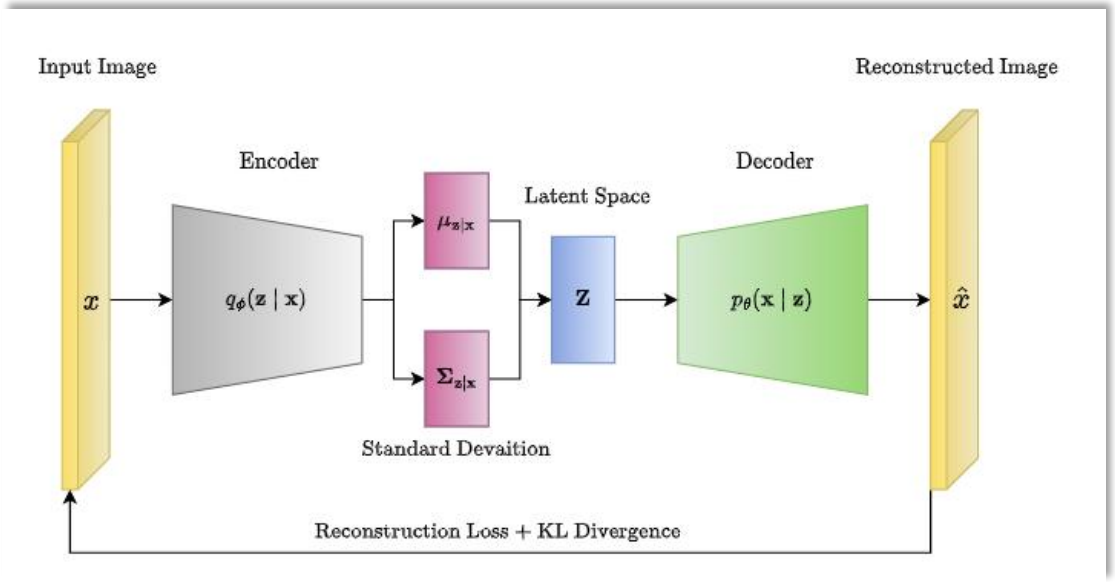
where J : dim. of z

Monte Carlo method (L2 norm)

$$\frac{1}{L} \sum_{l=1}^{L} \left(\log p_\theta\left(x|z^{(l)}\right)\right)$$

where $z^{(l)} \sim q_\phi(z|x)$

Sampling



https://www.youtube.com/watch?v=KYA-GEhObIs&list=PL0oFI08O71gKjGhaWctTPvvM7_cVzsAtK&index=9

Loss function (2/3)

- **Method 2**

ELBO (Evidence Lower Bound)

$$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$$

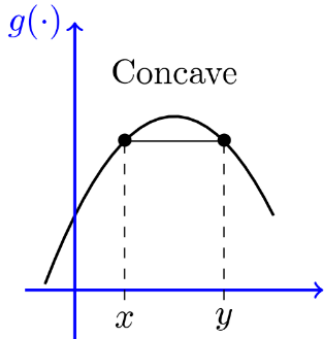
Marginalization → $$p(x) = \int p(x,z)\,dz = \int p(x|z)p(z)\,dz$$

$$\log p(x) = \log \int p(x,z)\,dz = \log \int q(z|x)\,\frac{p(x,z)}{q(z|x)}\,dz$$

Expectation → $$= \log \mathbb{E}_{q(z|x)}\left[\frac{p(x,z)}{q(z|x)}\right]$$

Jensen's inequality → $$\geq \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$$

ELBO → $$\mathcal{L}(q) := \mathbb{E}_{q(z|x)}\left[\log \frac{p(x,z)}{q(z|x)}\right]$$



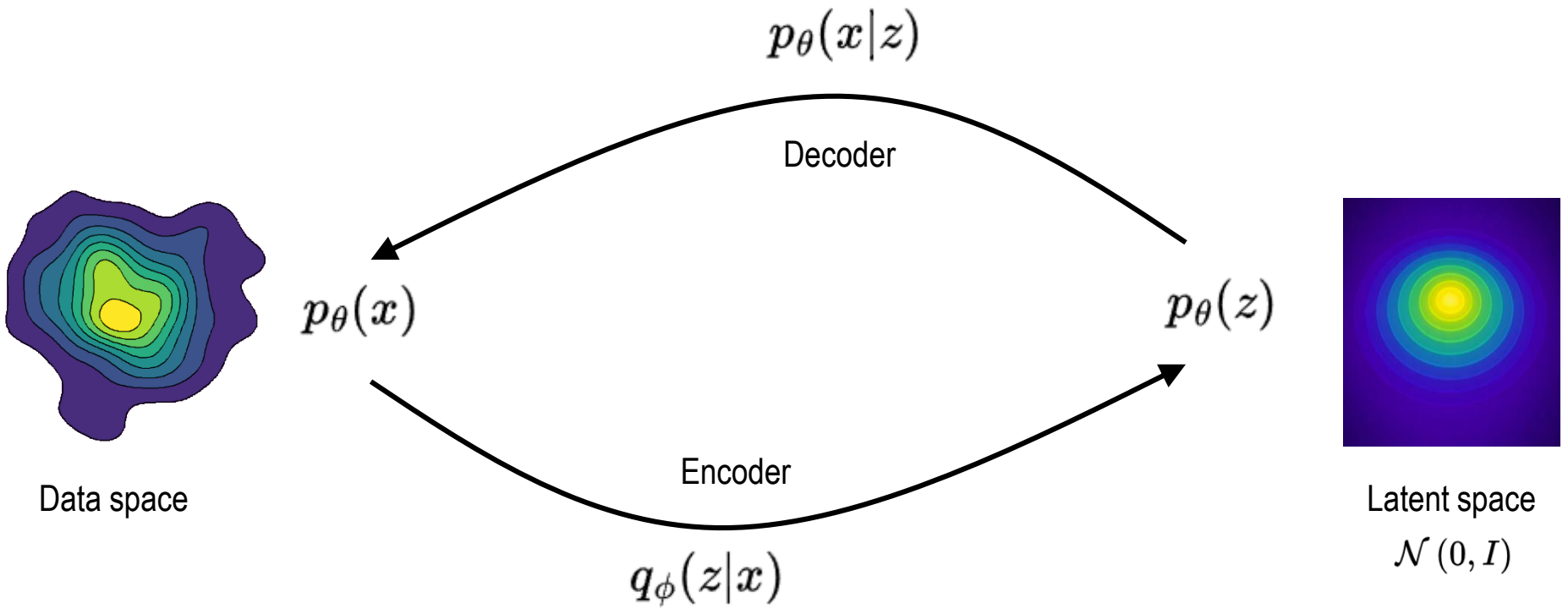
$$= \mathbb{E}_{q(z|x)}[\log p(x|z) + \log p(z) - \log q(z|x)]$$

$$= \underbrace{\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{reconstruction term}} \uparrow - \underbrace{\mathbb{E}_{q(z|x)}\left[\log \frac{q(z|x)}{p(z)}\right]}_{D_{\mathrm{KL}}(q(z|x)\|p(z))} \downarrow$$

Loss function (3/3)

- **Meaning of the loss function**, i.e., ELBO (Evidence Lower Bound)
  - Regularization → How close is the latent space to the prior (i.e., Standard Normal Distribution) ?
  - Reconstruction accuracy → How well the decoder reproduces the input ?

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}^{(i)}) = \underbrace{-D_{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})||p_{\boldsymbol{\theta}}(\mathbf{z}))}_{\text{Regularization}} + \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}^{(i)})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}|\mathbf{z})\right]}_{\text{Reconstruction accuracy}}$$

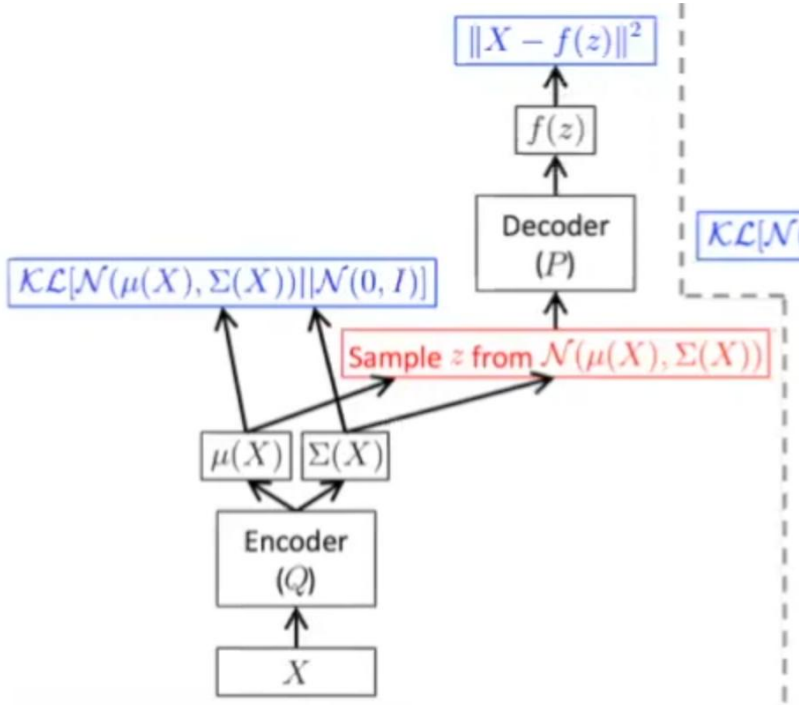


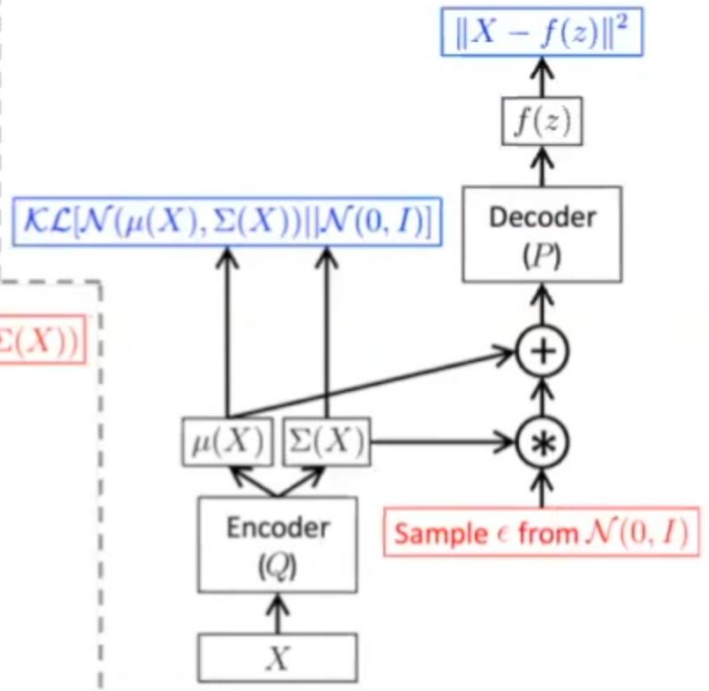
Parameterization trick

- **Sampling process is not differentiate** → Backpropagation is unavailable !

$$z^{(l)} \sim q_\phi(z|x)$$

$$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon, \quad \text{where } \epsilon \sim N(0,1)$$





$$\begin{aligned}
&\frac{\partial}{\partial W} \mathbb{E}_{q_\phi(z|x)} \left[\log p_\theta(x|z)\right] \\
&= \frac{\partial}{\partial W} \mathbb{E}_{\epsilon \sim N(0,1)} \left[\log p_\theta(x|\mu_q + \sigma_q \cdot \epsilon)\right] \\
&= \mathbb{E}_{\epsilon \sim N(0,1)} \left[\frac{\partial}{\partial W} \log p_\theta(x|\mu_q + \sigma_q \cdot \epsilon)\right]
\end{aligned}$$

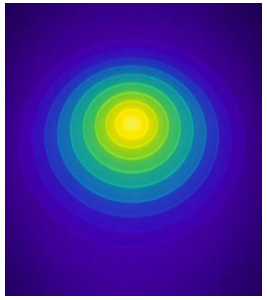
Now, differentiable !

https://www.youtube.com/watch?v=KYA-GEhObIs&list=PL0oFI08O71gKjGhaWctTPvvM7_cVzsAtK&index=9

Experiments

Latent space walking





Dependency on the dimension of the latent space



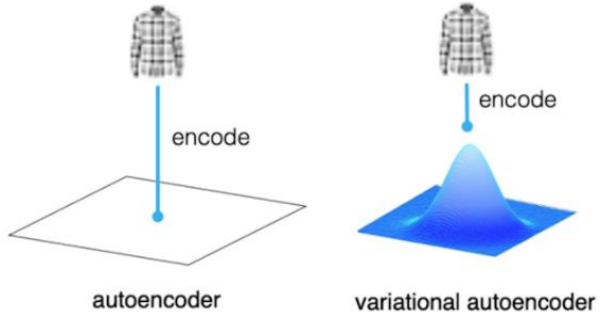
(a) 2-D latent space



(d) 20-D latent space

Summary

- **Limitations**
  - Often produces blurry images due to Gaussian assumptions
  - Hard to balance between reconstruction and regularization
  - Struggle with complex multimodal data distributions

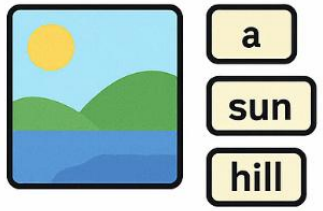


- **Auto-Encoder vs. VAE**

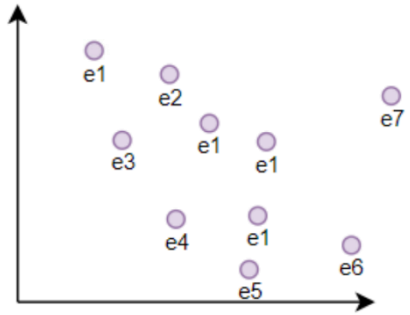
	Auto-Encoder	Variational Auto-Encoder
Goal	Compress and reconstruction	Data generation
Training	Minimize reconstruction error	Maximize ELBO
Latent representation	Deterministic	Probabilistic
Generative capability	Low (Only reconstruction)	High (Generate new, unseen images)
Applications	Dimension reduction, Denoising	Image generation, anomaly detection
Output quality	Sharp but limited diversity	More diverse but sometimes blurry

[Vector Quantized-VAE]

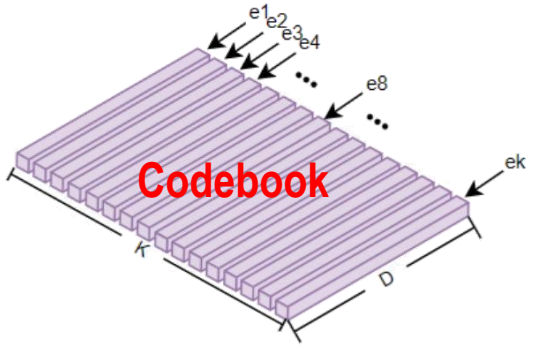
- **Motivation**
  - VAEs produce blurry reconstructions
    → Continuous latent space causes averaging of pixel uncertainties
  - Need more structured discrete representations
    → Discrete codes can capture high-level semantics more clearly.
  - A foundation for vision-language tokenization
    → Images become token sequences like words, enabling models like DALL-E.



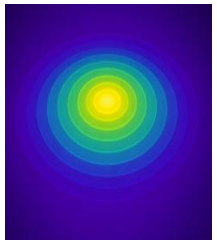
- **Goal**
  - Improve perceptual image quality by introducing discrete latent representations



Discrete latent representation



Continuous latent representation



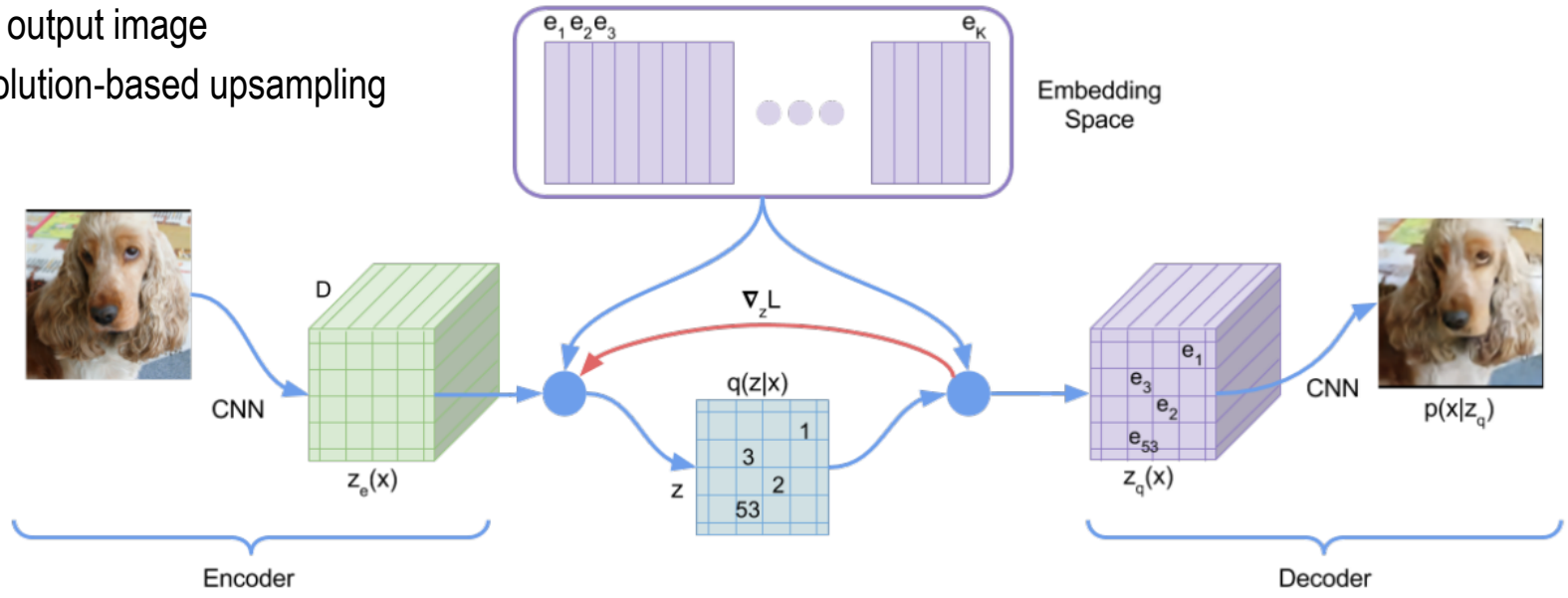


https://gaussian37.github.io/dl-concept-vae/

Blurry outputs of VAE

Architecture

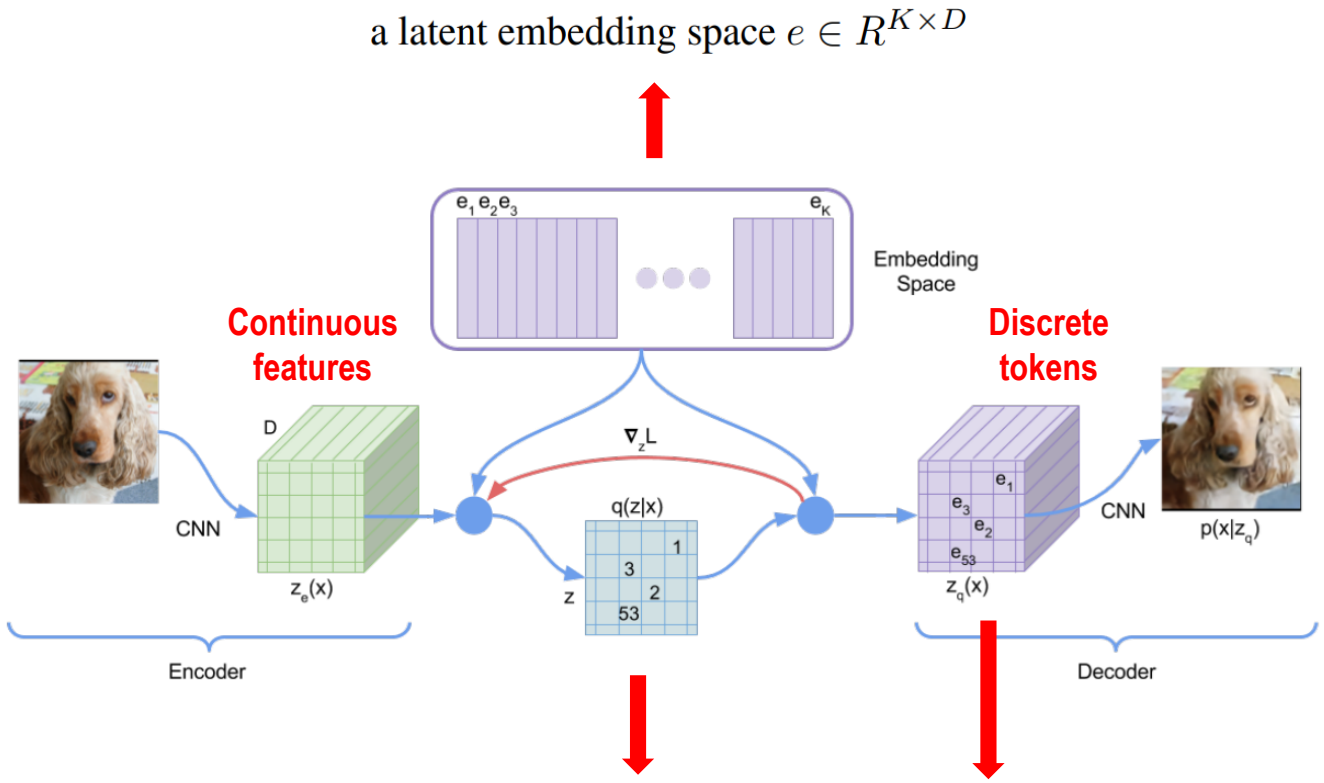
- **Encoder** (CNN)
  - Converts an input image into low-resolution latent features → Continuous latent vectors
- **Codebook** (Discrete Latent Space)
  - Represents a dictionary of visual prototypes
  - Each code is a compressed feature token representing a local patch
  - Codebook stores *K* learnable embedding vectors
- **Vector Quantization** (Codebook Lookup)
  - Replaces each encoder latent vector with the nearest codebook entry
    → Transforms continuous features into discrete indices
- **Decoder** (CNN)
  - Quantized latent vectors → Reconstructs the output image
  - Learns to restore details and texture with convolution-based upsampling



https://lilianweng.github.io/posts/2018-08-12-vae/

Representation learning

- Converting continuous values into a finite set of discrete representations.
  → **Building a discrete latent space using a learned codebook**
- Enabling neural networks to learn a **symbolic vocabulary of visual concepts**



$$q(z = k|x) = \begin{cases} 1 & \text{for } k = \text{argmin}_j \|z_e(x) - e_j\|_2, \\ 0 & \text{otherwise} \end{cases}$$

$$z_q(x) = e_k, \quad \text{where} \quad k = \text{argmin}_j \|z_e(x) - e_j\|_2$$

Prior learning

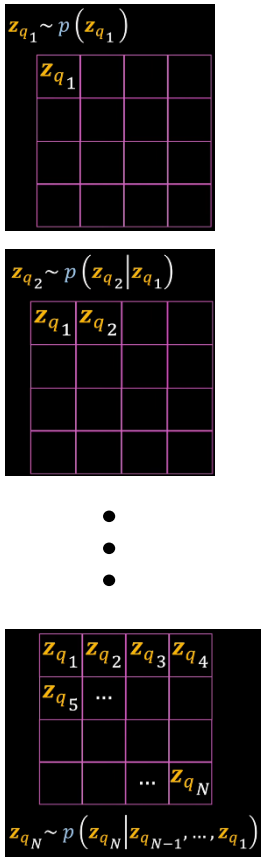
- **Auto-regressive quantization**
  - Direct independent quantization may produce inconsistent results
    → Artifacts, broken textures, missing long-range details
  - The quantized latent tokens should depend on neighboring tokens
    → Captures global structure and spatial coherence

Autoregressive modeling enforces
context-aware token selection !

Ancestral sampling

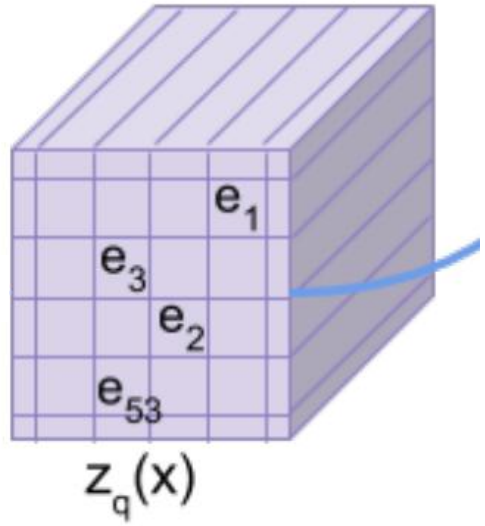
$$z_{q_1} \sim p(z_{q_1})$$
$$\downarrow$$
$$z_{q_2} \sim p(z_{q_2} | z_{q_1})$$
$$\downarrow$$
$$z_{q_3} \sim p(z_{q_3} | z_{q_2}, z_{q_1})$$
$$\downarrow$$
$$\dots$$
$$\downarrow$$
$$z_{q_N} \sim p(z_{q_N} | z_{q_{N-1}}, \dots, z_{q_2}, z_{q_1})$$

$$p(z_q) = \prod_i p(z_{q,i} \mid z_{q,<i})$$



PixelCNN

$$\mathcal{L}_{\text{PixelCNN}} = -\sum_{q=1}^{N}\sum_{k=1}^{K} \mathbf{1}[z_q = k] \log p_\theta(z_q = k \mid z_{<q})$$



https://www.youtube.com/watch?v=yQvELPjmyn0&t=923s

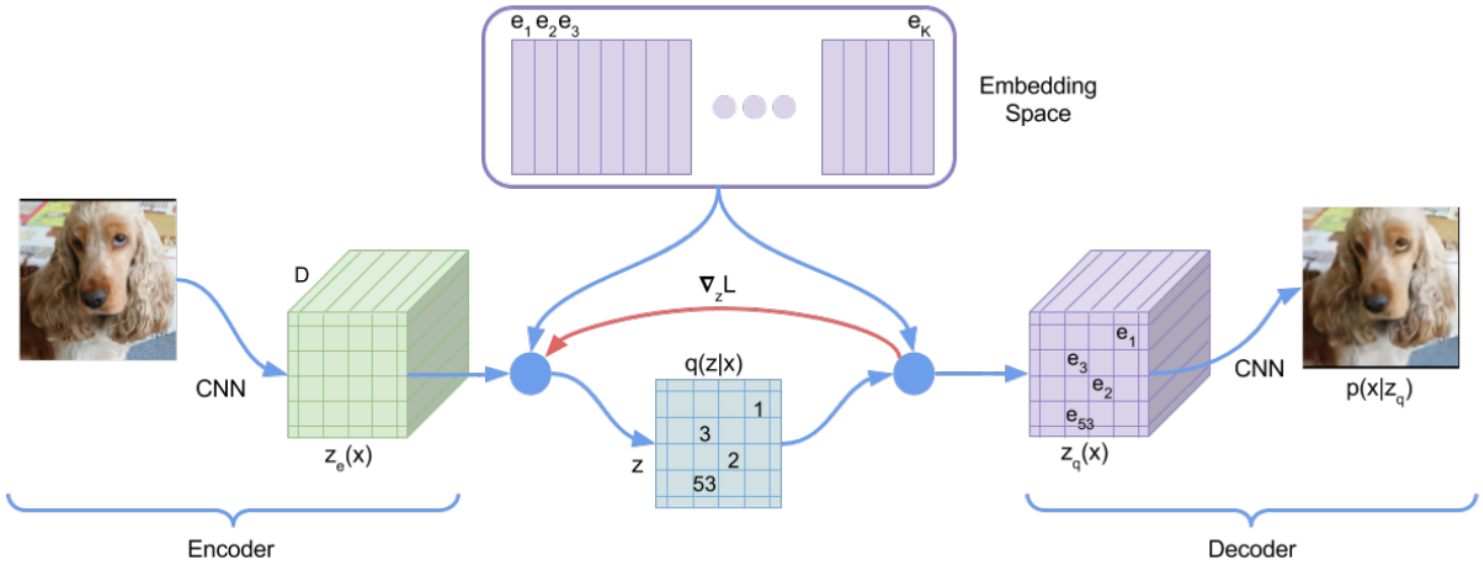
Loss

- **Reconstruction loss** → Improve output image fidelity
- **Codebook loss** → Update embedding code vectors to match encoder outputs
- **Commitment loss** → Encourage encoder to commit to discrete codes

sg : "stopgradient operator"
Identity at forward computation time and has zero partial derivatives
→ constraining its operand to be a non-updated constant

$$L = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2,$$

Reconstruction loss (Encoder, Decoder) | **Codebook loss (Codebook)** | **Commitment loss (Encoder)**



STE (Straight-Through Estimator)

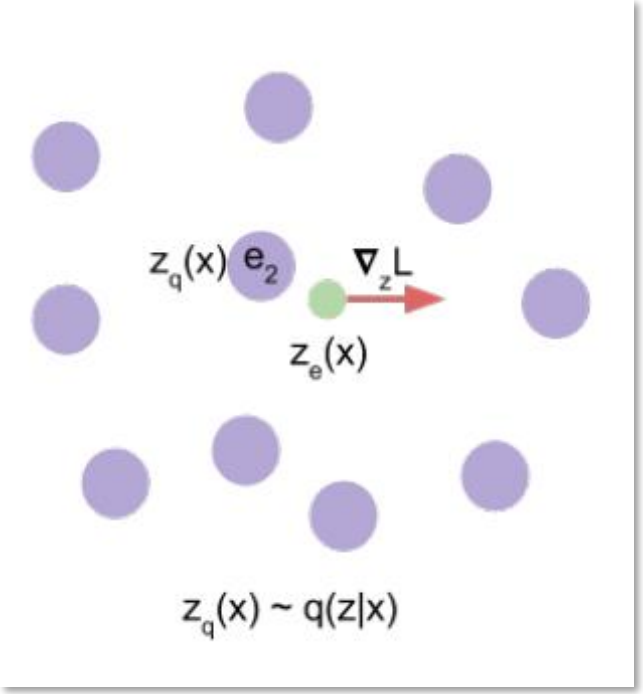
Neural Discrete Representation Learning, 2017

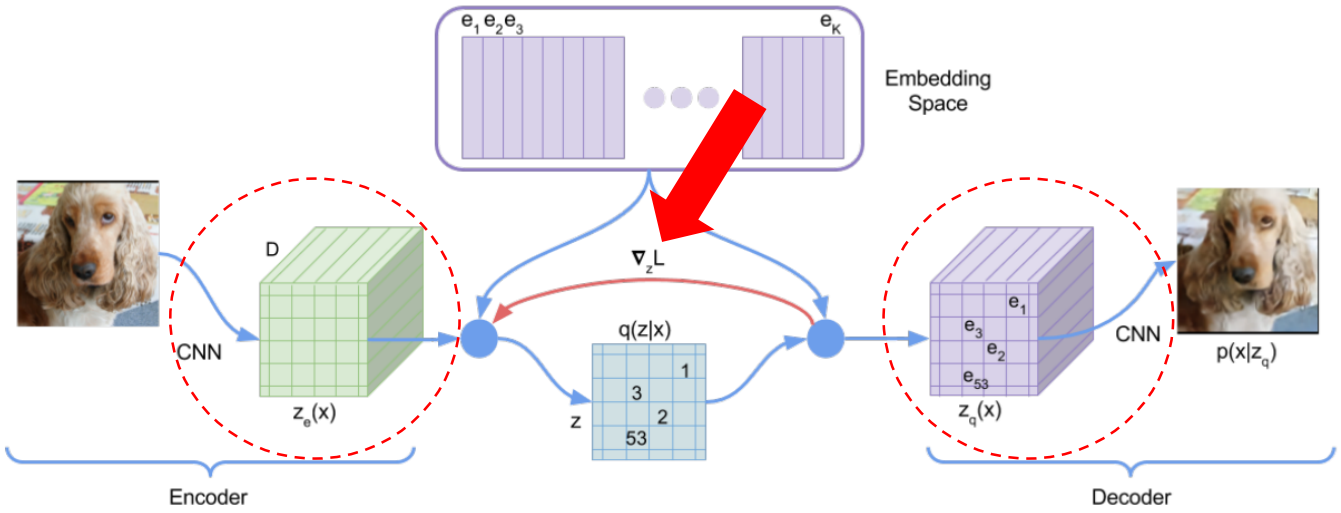
- **Forward pass** → Nearest neighbor lookup

$$z_q(x) = e_k, \quad \text{where} \quad k = \text{argmin}_j \|z_e(x) - e_j\|_2$$

- Problem in **backward pass**
  - 'argmin' is the non-differential operation
- **Solution**
  - Copying the gradients from the decoder input directly to the encoder output

$$\text{if } z_e(x) \approx z_q(x), \quad \frac{\partial L}{\partial z_e(x)} = \frac{\partial L}{\partial z_q(x)}$$





Experiments

- **Reconstruction**
  - Sharper and more realistic reconstruction than VAE
    → Solves the blurry problem of VAE
- **Generation**
  - Diverse and coherent new images

Discretization is a powerful inductive bias
for various modalities !

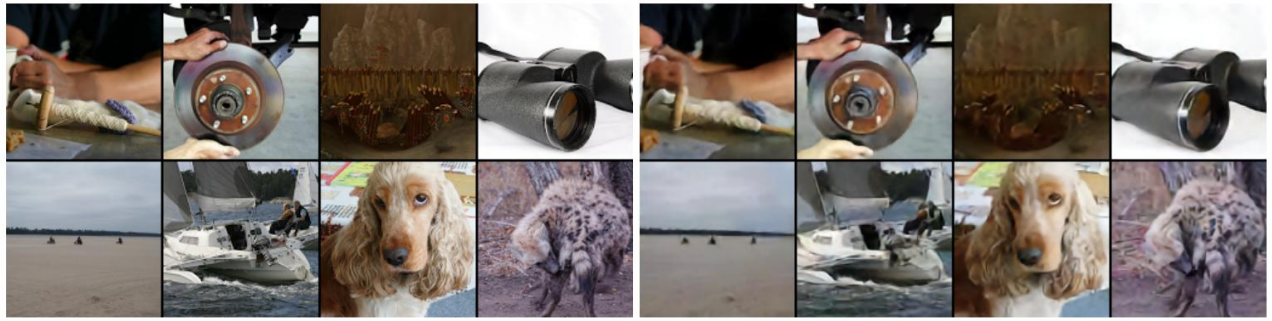


Figure 2: Left: ImageNet 128x128x3 images, right: reconstructions from a VQ-VAE with a 32x32x1 latent space, with K=512.

Reconstruction

Generation

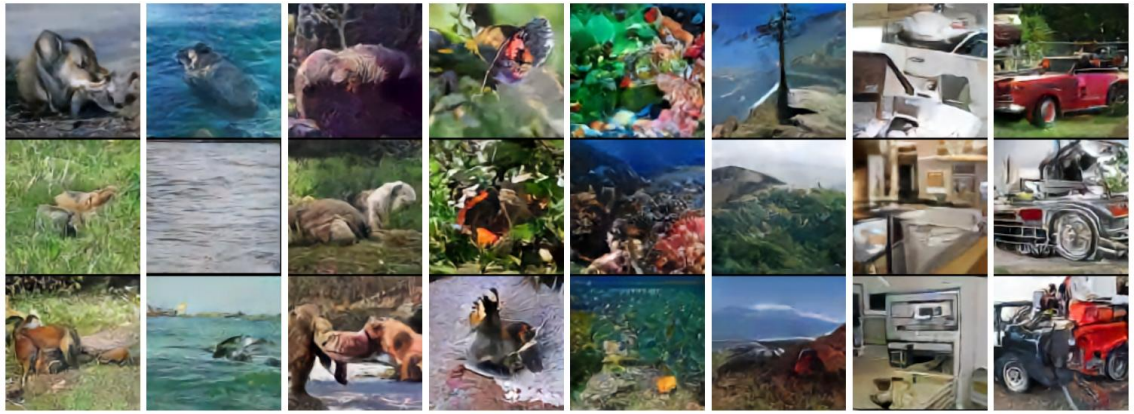


Figure 3: Samples (128x128) from a VQ-VAE with a PixelCNN prior trained on ImageNet images. From left to right: kit fox, gray whale, brown bear, admiral (butterfly), coral reef, alp, microwave, pickup.

Summary

- **Limitations**
  - Codebook inefficiency → Codebook collapse
  - Limited resolution detail → Quantization error
  - Reliance on approximate gradient estimation

- **VAE vs. VQ-VAE**

	VAE	**VQ-VAE**
Latent space	Continuous (Gaussian)	Discrete (Codebook-based)
Image quality	Blurry reconstruction due to KL regularization	Sharp – clear edges and textures
Performance	Moderate	Higher fidelity
Contribution	Parameterization trick	Foundation for DALL·E and VQGAN, etc.

Diffusion Model

[DDPM]

- **DDPM (Denoising Diffusion Probabilistic Models)**

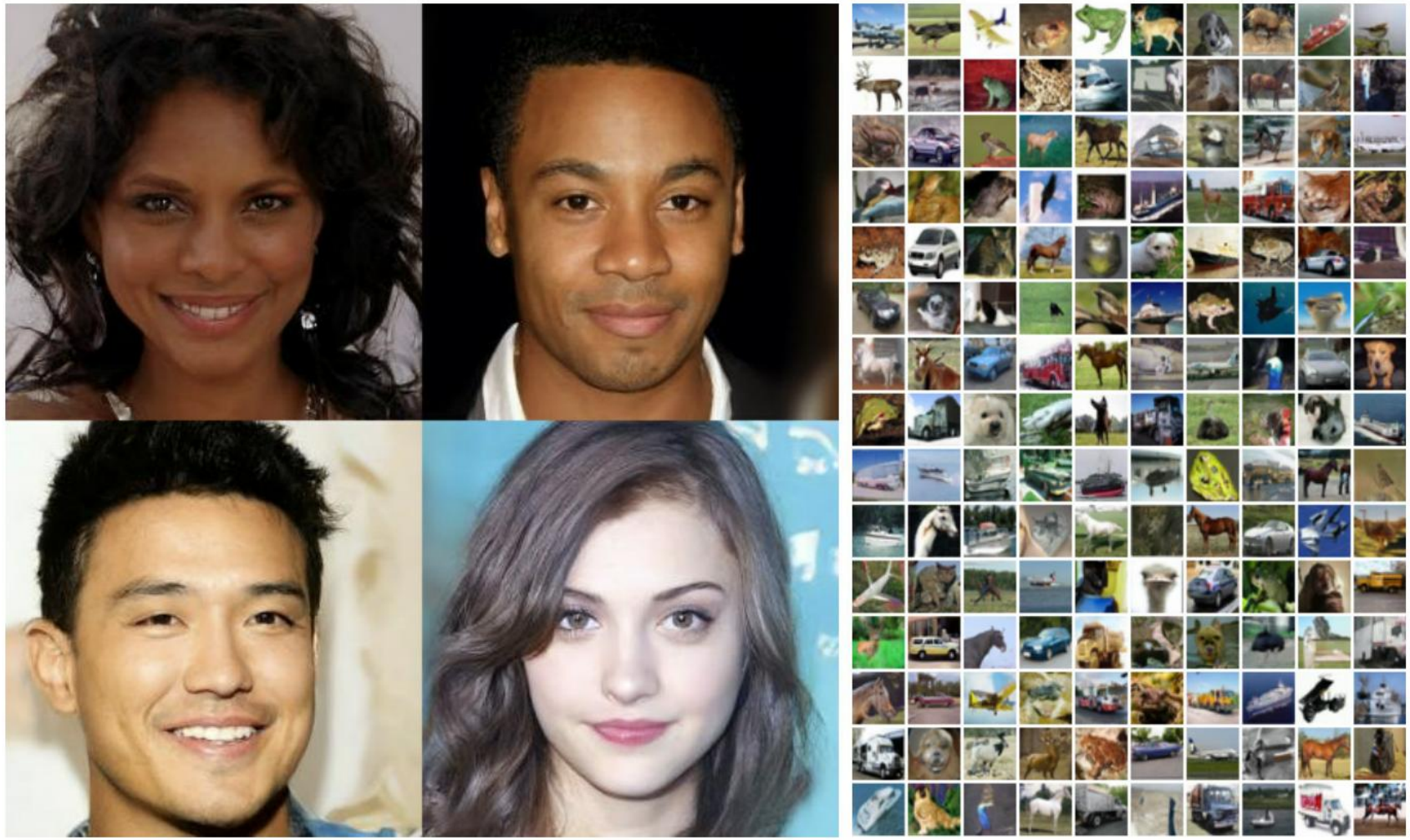
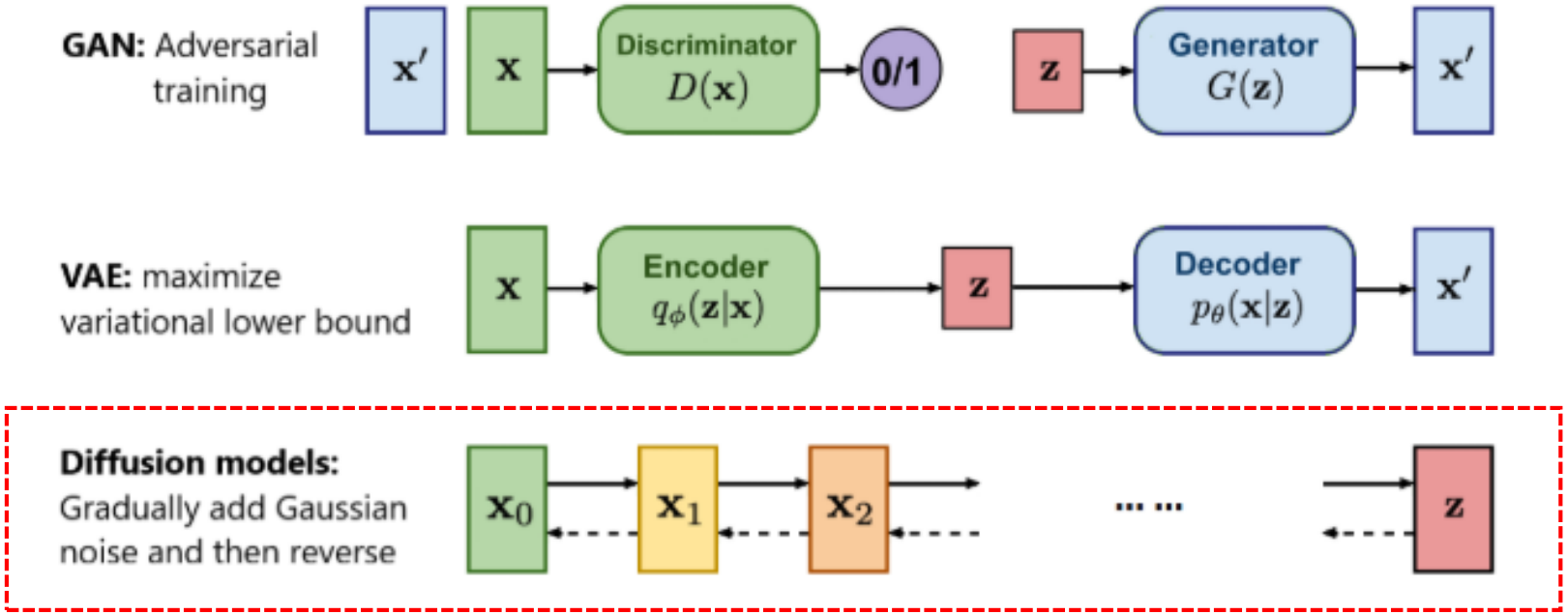


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

Overview

- **Motivation (Limitation of the conventional approaches)**
  - GANs
    - Training instability → Mode collapse
  - VAEs
    - Stable training
    - But, often produce blurry samples
- **Goal**
  - Learn a complex data distribution by reversing a gradual Gaussian noising process (i.e., diffusion process)

Diffusion models convert the image generation into a sequence of simple denoising steps !



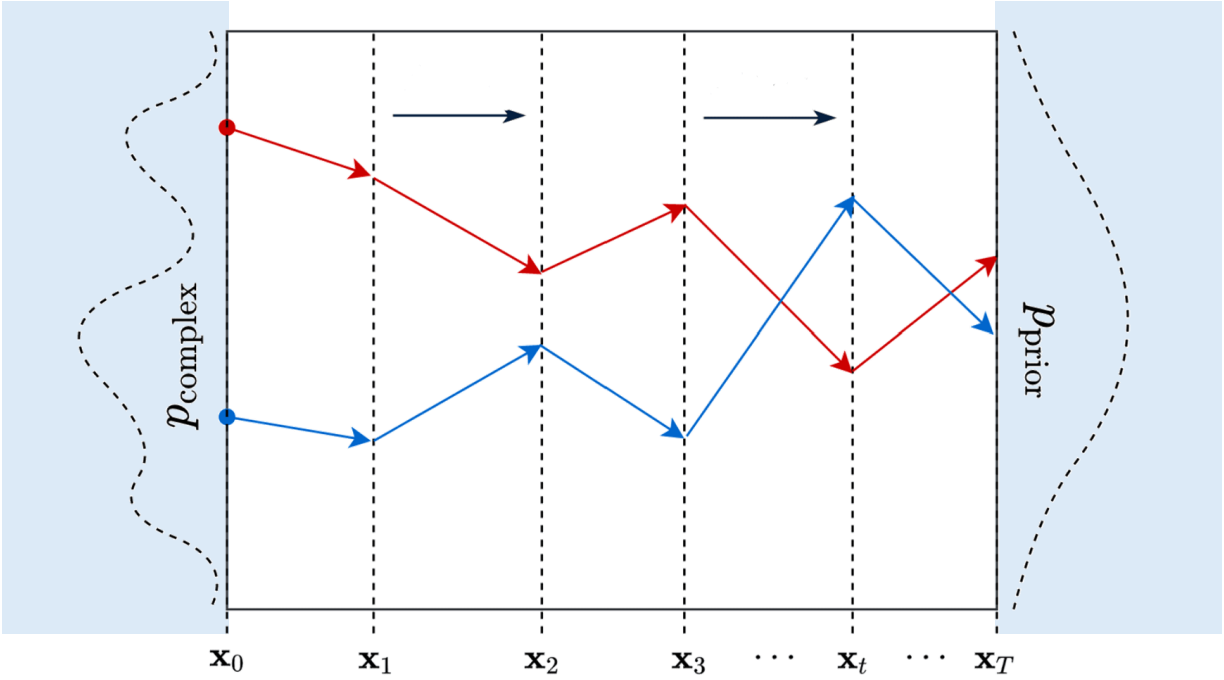
https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Diffusion (1/2)

- **What is the diffusion process ?**
  - A stochastic Markov process having continuous sample path (i.e., no jumps)
  - In the context of statistics,
    - Repeated application of a transition kernel on the samples of any distribution would lead to samples from the simple prior
    - Transforming a complex distribution to a simple (predefined) distribution prior on the same domain



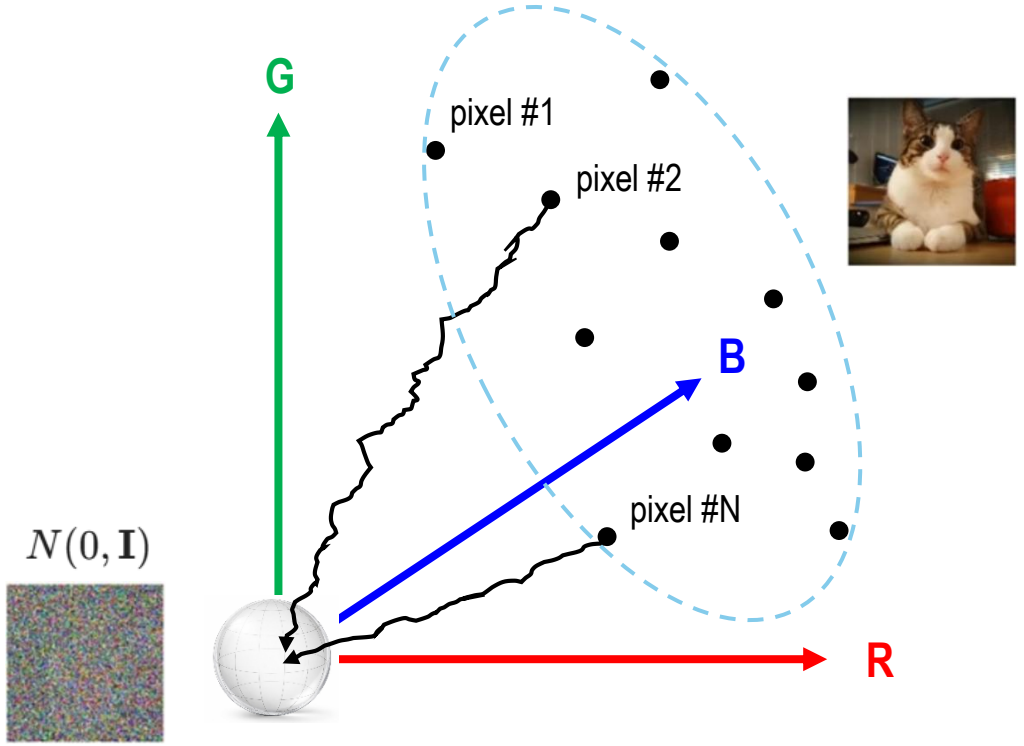
https://www.accessscience.com/content/article/a194200



https://learnopencv.com/denoising-diffusion-probabilistic-models/

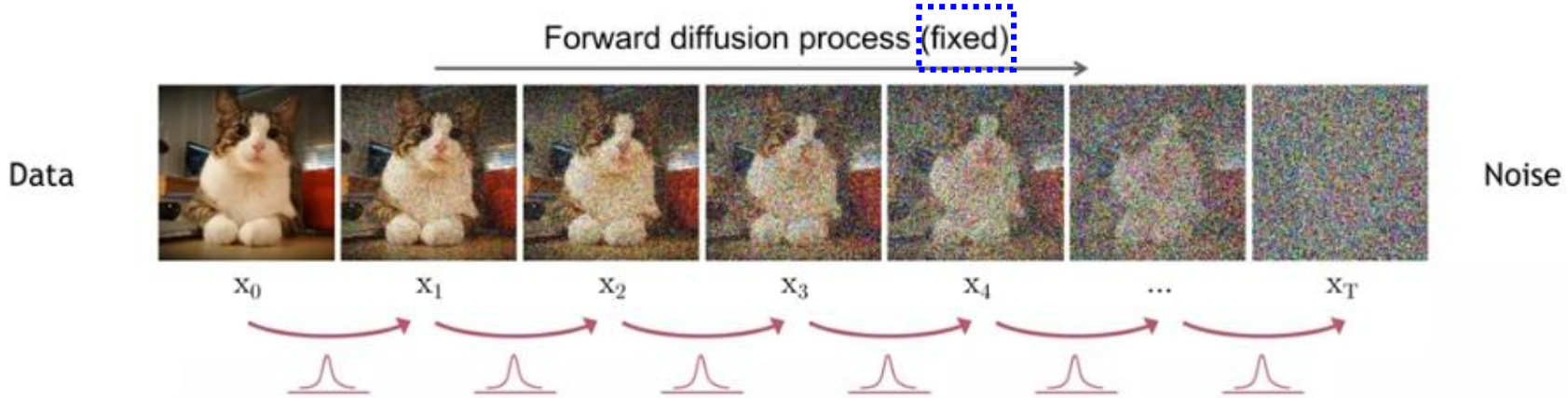
Diffusion (2/2)

- **The Gaussian trick**
  - Targeting "Isotropic Gaussian Distribution" in "pixel space"
    - Tractable starting point for generation
    - Not latent space but pixel space to avoid the blurriness
- **Local movement**
  - During a "small time period", pixel moves only a fraction from its previous state in a "random direction"
    → So if we can predict (i.e., model) this stochastic process
    → Then, we can learn to reverse them with high precision

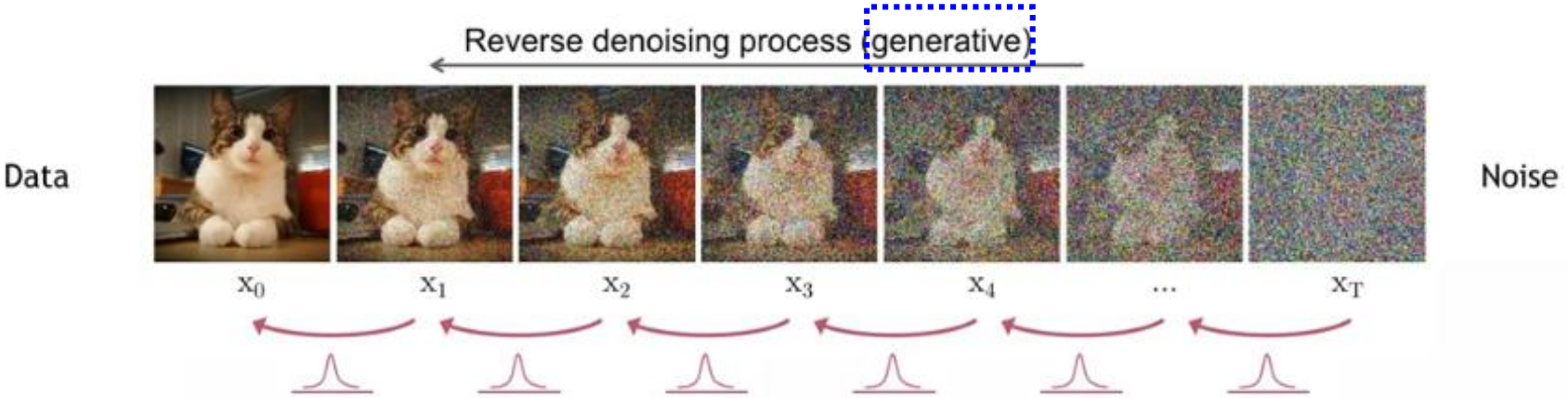


Overall process

- **Forward process** (Diffusion / Noising)
  - Gradually corrupt data with Gaussian noise



- **Reverse process** (Generation / Denoising)
  - Learn to reverse the noise using a neural network



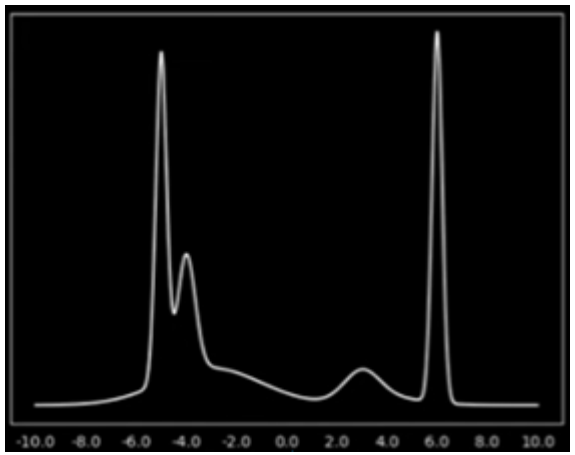
https://velog.io/@ryuseunghan/Diffusion-Model

Forward diffusion kernel

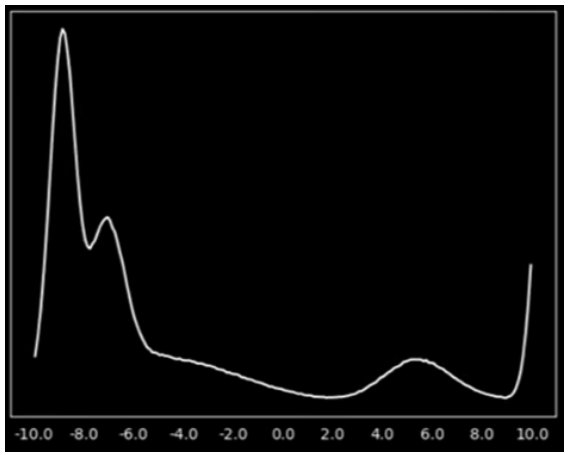
- **Conceptual definition**
  - Defines the transition probability of adding noise at a single time step
  - In the last time step, we need to have normal Gaussian distribution, i.e., $\mathcal{N}(0, \mathrm{I})$
- **Key characteristics**
  - Markovian property
  - Information loss – original signal is gradually destroyed

$$x_t = a\, x_{t-1} + b\, \mathcal{N}(0, \mathrm{I})$$

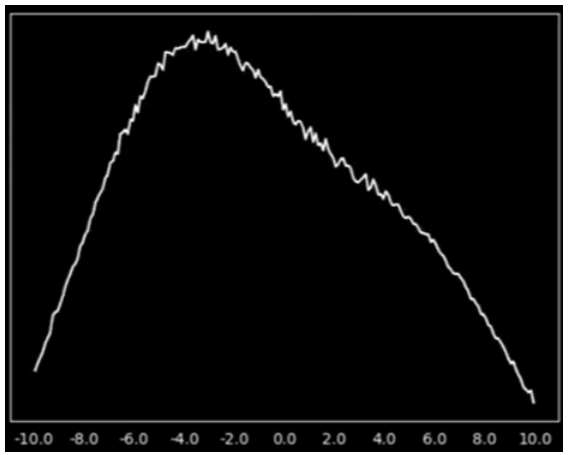
$a = 1.1\ ,\ b = 0.1$ $\quad$ $a = 0.999\ ,\ b = 1$ $\quad$ $a = \sqrt{0.99}\ ,\ b = \sqrt{0.01}$



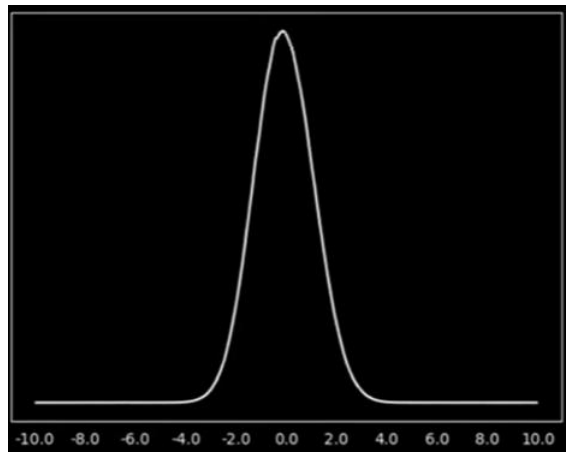
x_0



$t \rightarrow$



$t \rightarrow$



$t \rightarrow$

https://www.youtube.com/watch?v=H45lF4sUgiE

Noise schedule (1/2)

$$x_t = \sqrt{1-\beta}\, x_{t-1} + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})$$

$$x_T = \sqrt{1-\beta}\, x_{T-1} + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})$$
$$x_T = \sqrt{1-\beta}\, \left(\sqrt{1-\beta}\, x_{T-2} + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})\right) + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})$$
$$x_T = \sqrt{1-\beta}^2\, x_{T-2} + \sqrt{1-\beta}\sqrt{\beta}\, \mathcal{N}(0, \mathrm{I}) + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})$$
$$\cdots$$
$$x_T = \underbrace{\sqrt{1-\beta}^T}_{0}\, x_0 + \underbrace{\cdots + \sqrt{1-\beta}^2 \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I}) + \sqrt{1-\beta}\sqrt{\beta}\, \mathcal{N}(0, \mathrm{I}) + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})}_{\mathcal{N}(0, \mathrm{I})}$$

$$X \sim \mathcal{N}(0, \mathrm{I})$$
$$Y = aX$$
$$E[Y] = aE[X] = 0$$
$$V[Y] = a^2 V[X] = a^2$$

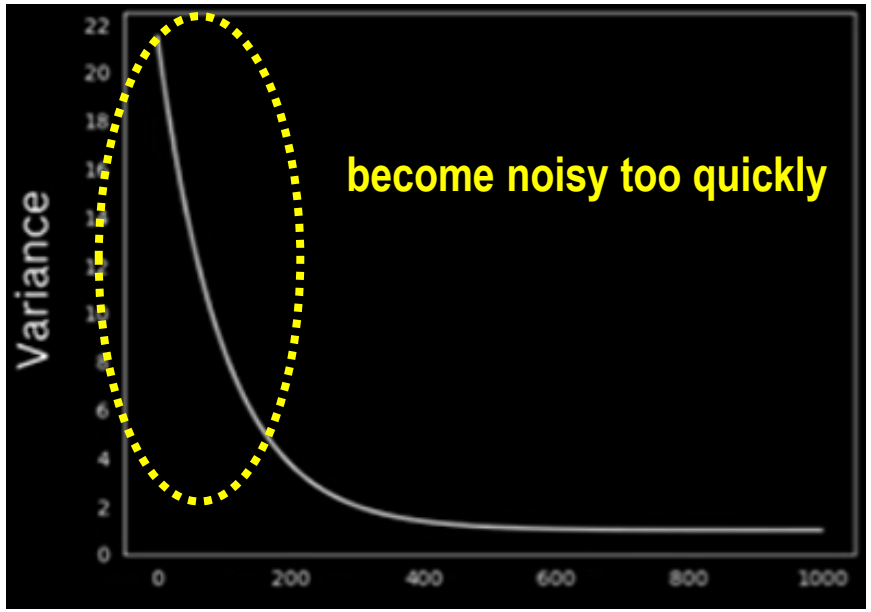
$$\sum_{n=0}^{\infty} ar^n = \frac{a}{1-r}, \quad |r| < 1$$

Noise schedule (2/2)

- **Linear noise schedule**
  - Increases β linearly from small to larger values over time steps
    → Gradually increasing noise

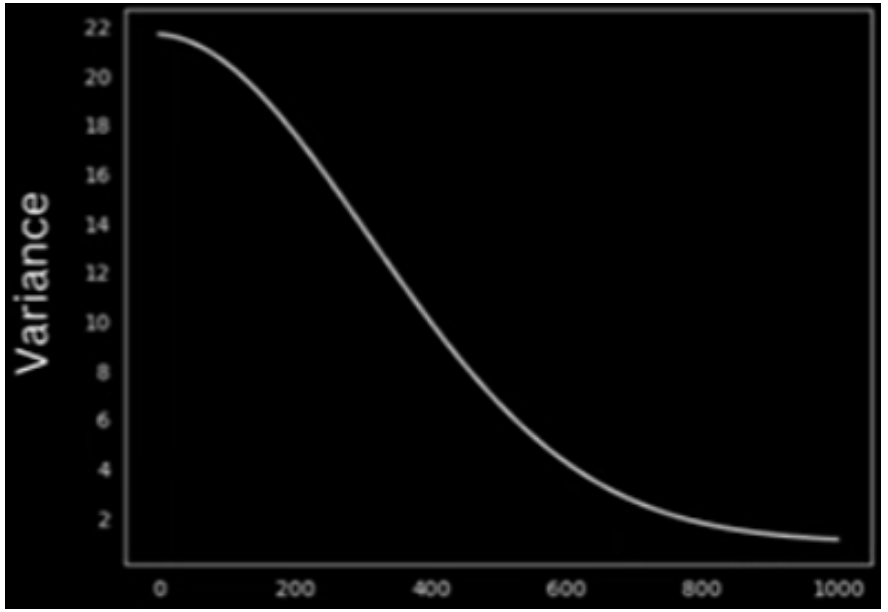
Constant noise schedule

$$x_t = \sqrt{1-\beta}\, x_{t-1} + \sqrt{\beta}\, \mathcal{N}(0, \mathrm{I})$$



Linear noise schedule

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \mathcal{N}(0, \mathrm{I})$$
$$\beta_1 = 0.0001, \cdots, \beta_T = 0.02$$



https://www.youtube.com/watch?v=H45lF4sUgiE

Forward diffusion process

- **Forward diffusion process is fixed.**
  - Noisy version x_t can be computed analytically from x_0 using the noise schedule β_t
  - No neural network is involved in the forward direction $\rightarrow$ Purely a stochastic and predefined Gaussian process

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \mathcal{N}(0, \mathrm{I})$$ ⬅ **Reparameterization trick**

① $$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t, \text{ where } \alpha_t = 1-\beta_t$$ ⬅ **From x_{t-1} to x_t**

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t$$

$$x_t = \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \epsilon_{t-1}\right) + \sqrt{1-\alpha_t}\, \epsilon_t$$

$$x_t = \sqrt{\alpha_t}\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t}\sqrt{1-\alpha_{t-1}}\, \epsilon_{t-1} + \sqrt{1-\alpha_t}\, \epsilon_t$$

$$x_t = \sqrt{\alpha_t\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\alpha_{t-1}}\, \epsilon$$

$$\cdots$$

$$x_t = \sqrt{\alpha_t\alpha_{t-1}\cdots\alpha_1}\, x_0 + \sqrt{1-\alpha_t\alpha_{t-1}\cdots\alpha_1}\, \epsilon$$

② $$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \text{ where } \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$$ ⬅ **From x_0 to x_t**



x_0

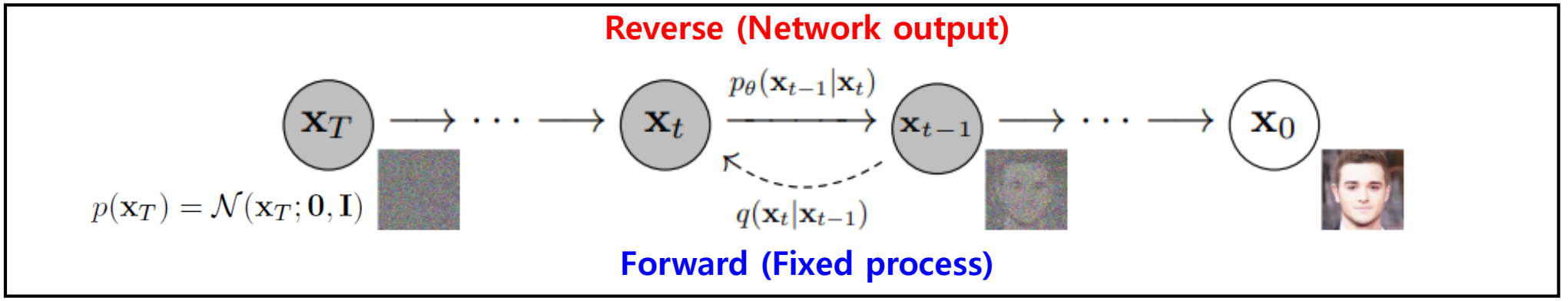
⬇



x_t

Loss (1/4)

- **Bound of the log likelihood**



$$\log p(x_0) = \log \int p(x_{0:T})\, dx_{1:T}$$

$$= \log \int p(x_{0:T}) \frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)}\, dx_{1:T}$$

$$= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

$$\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

$$\log \frac{p(x_{0:T})}{q(x_{1:T}|x_0)} = \log \frac{p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)}{q(x_1|x_0) \prod_{t=2}^{T} q(x_t|x_{t-1}, x_0)}$$

$$= \log \frac{p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)}{q(x_T|x_0) \prod_{t=2}^{T} q(x_{t-1}|x_t, x_0)}$$

$$= \log \frac{p(x_T) p_\theta(x_0|x_1) \prod_{t=2}^{T} p_\theta(x_{t-1}|x_t)}{q(x_T|x_0) \prod_{t=2}^{T} q(x_{t-1}|x_t, x_0)}$$

$$= \log \frac{p(x_T)}{q(x_T|x_0)} + \log p_\theta(x_0|x_1) + \sum_{t=2}^{T} \log \frac{p_\theta(x_{t-1}|x_t)}{q(x_{t-1}|x_t, x_0)}$$

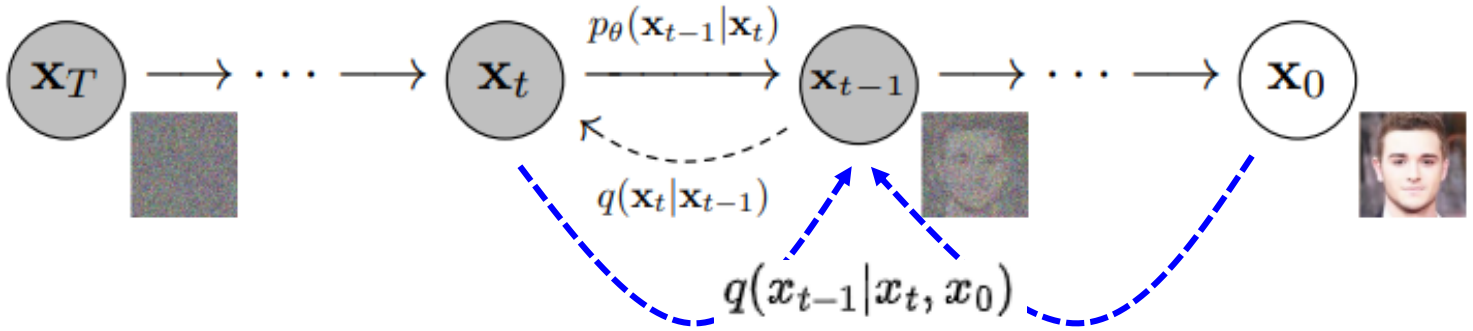
Network

Target

Not dependent on θ

Very small

Loss (2/4)



$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)\, q(x_{t-1}|x_0)}{q(x_t|x_0)}$$

$$= \frac{\mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)\mathrm{I})\ \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1-\bar{\alpha}_{t-1})\mathrm{I})}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_{t-1})\mathrm{I})}$$

⇦ by ① and ②

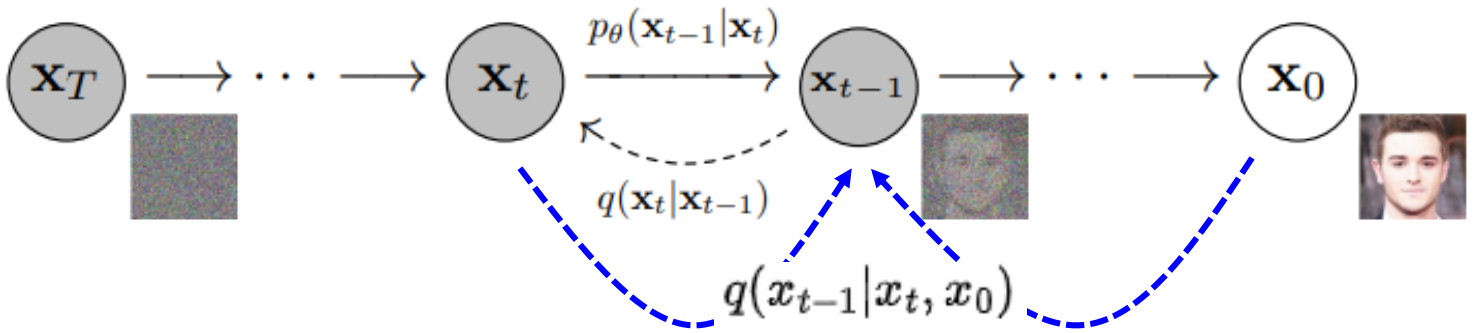
$$= \mathcal{N}\left(x_{t-1};\ \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}x_t\ +\ (1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}x_0}{1-\bar{\alpha}_t},\ \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

$$= \mathcal{N}\left(x_{t-1};\ \mu_q(x_t), \Sigma_q(t)\right)$$

$$\updownarrow \quad \dashrightarrow \quad D_{KL}(q,p) = \frac{1-\bar{\alpha}_t}{2(1-\alpha_t)(1-\bar{\alpha}_{t-1})}\|\mu_q - \mu_\theta\|_2^2$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1};\ \mu_\theta(x_t), \Sigma_q(t)\right)$$

Loss (3/4)

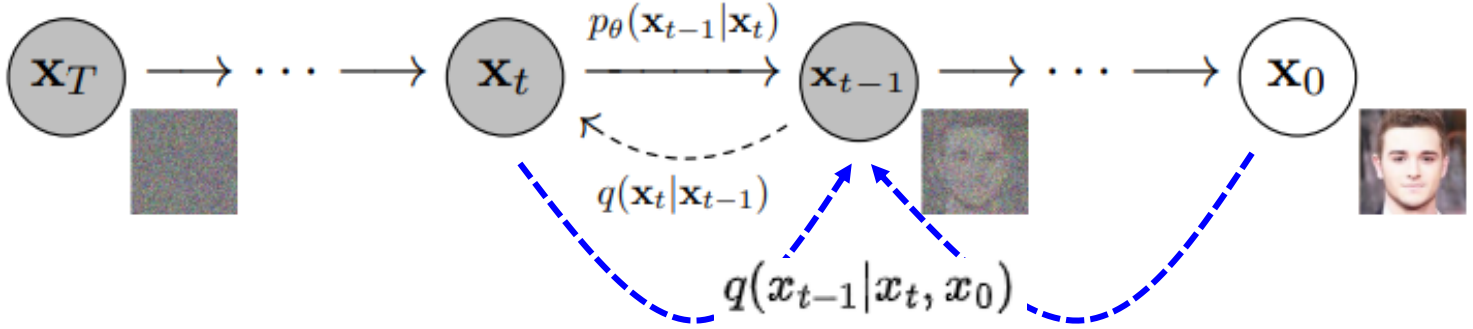


$$q(x_{t-1}|x_t, x_0) = \mathcal{N}\left(x_{t-1};\ \frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}x_t\ +\ (1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}x_0}{1-\overline{\alpha}_t},\ \frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}\mathrm{I}\right)$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1};\ \frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}x_t\ +\ (1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}x_\theta}{1-\overline{\alpha}_t},\ \frac{(1-\alpha_t)(1-\overline{\alpha}_{t-1})}{1-\overline{\alpha}_t}\mathrm{I}\right)$$

$$D_{KL}(q,p) = \frac{(1-\alpha_t)\overline{\alpha}_{t-1}}{2(1-\overline{\alpha}_t)(1-\overline{\alpha}_{t-1})}\|x_0 - x_\theta\|_2^2$$

Loss (4/4)



$$q(x_{t-1}|x_t,x_0) = \mathcal{N}\left(x_{t-1};\ \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}x_t\ +\ (1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}x_0}{1-\bar{\alpha}_t},\ \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

$$= \mathcal{N}\left(x_{t-1};\ \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}x_t\ +\ (1-\alpha_t)\sqrt{\bar{\alpha}_{t-1}}\frac{x_t-\sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}}{1-\bar{\alpha}_t},\ \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right) \quad \Leftarrow \quad x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon \quad x_0 = \frac{x_t-\sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$$

$$= \mathcal{N}\left(x_{t-1};\ \left(\frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t} + \frac{1-\alpha_t}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\right)x_t - \frac{(1-\alpha_t)\sqrt{1-\bar{\alpha}_t}}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\epsilon,\ \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1};\ \left(\frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t} + \frac{1-\alpha_t}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\right)x_t - \frac{(1-\alpha_t)\sqrt{1-\bar{\alpha}_t}}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\epsilon_\theta,\ \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

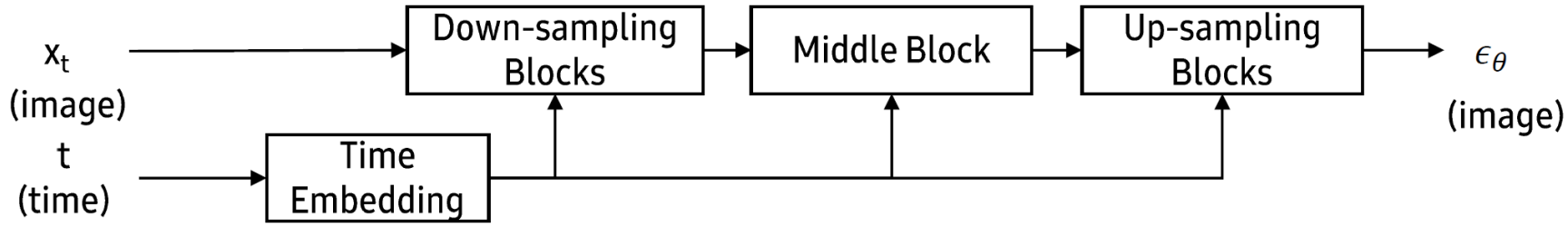
$$D_{KL}(q,p) = \frac{1-\bar{\alpha}_t}{2(1-\alpha_t)(1-\bar{\alpha}_{t-1})}\frac{(1-\alpha_t)^2}{\alpha_t(1-\bar{\alpha}_t)}\|\epsilon-\epsilon_\theta\|_2^2$$

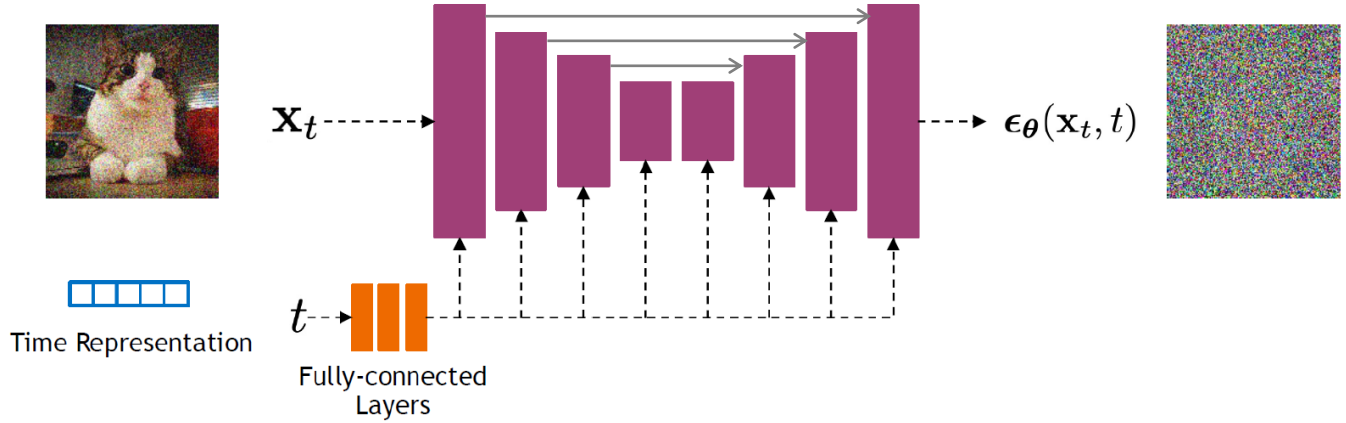
$$= \frac{1}{2\sigma_q^2}\frac{\beta_t^2}{\alpha_t(1-\bar{\alpha}_t)}\left\|\epsilon-\epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0+\sqrt{1-\bar{\alpha}_t}\epsilon,\ t)\right\|_2^2$$

$$(x_t, t) \rightarrow \boxed{\theta} \rightarrow \epsilon \rightarrow x_{t-1} = x_t - \epsilon$$

Network (θ) architecture

	Function	Components
Down-sampling blocks	Extracts high-level hierarchical features by progressively reducing spatial size	Residual blocks and Self-attentions layers
Middle block	Processes the most compressed latent representation	Residual block and Self-attention layer
Up-sampling blocks	Restores spatial resolution to match the original image size	Skip connections & Transposed convolutions or Up-sampling layers



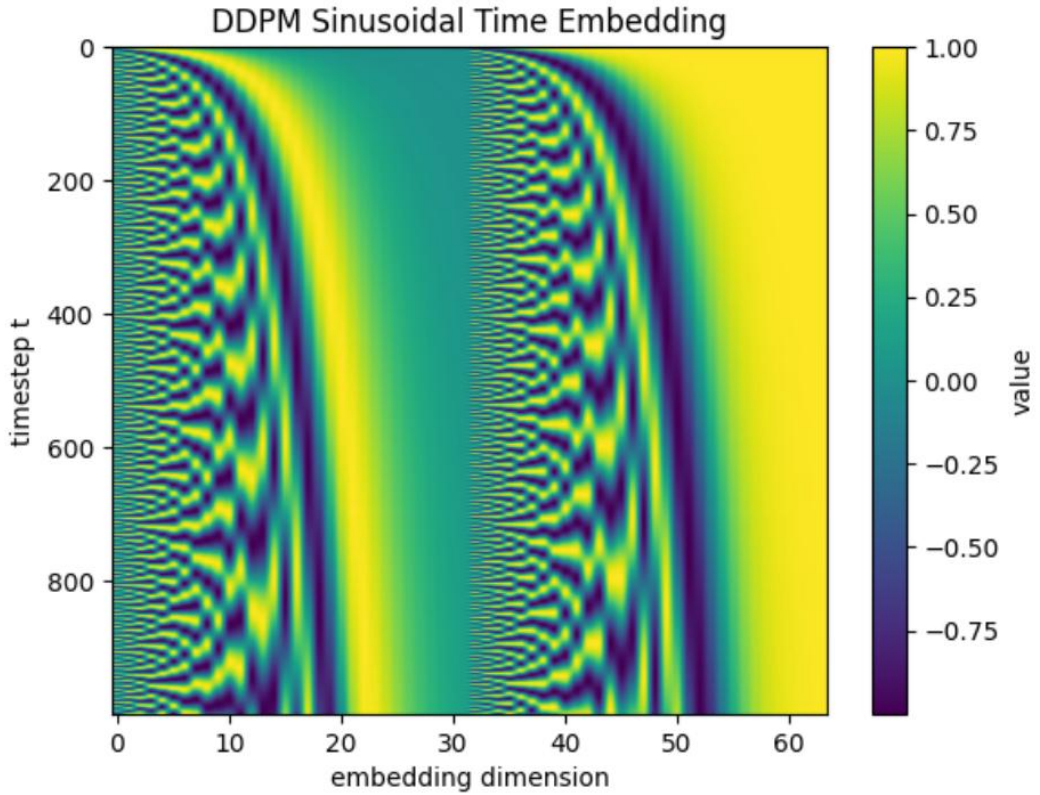


Time embedding

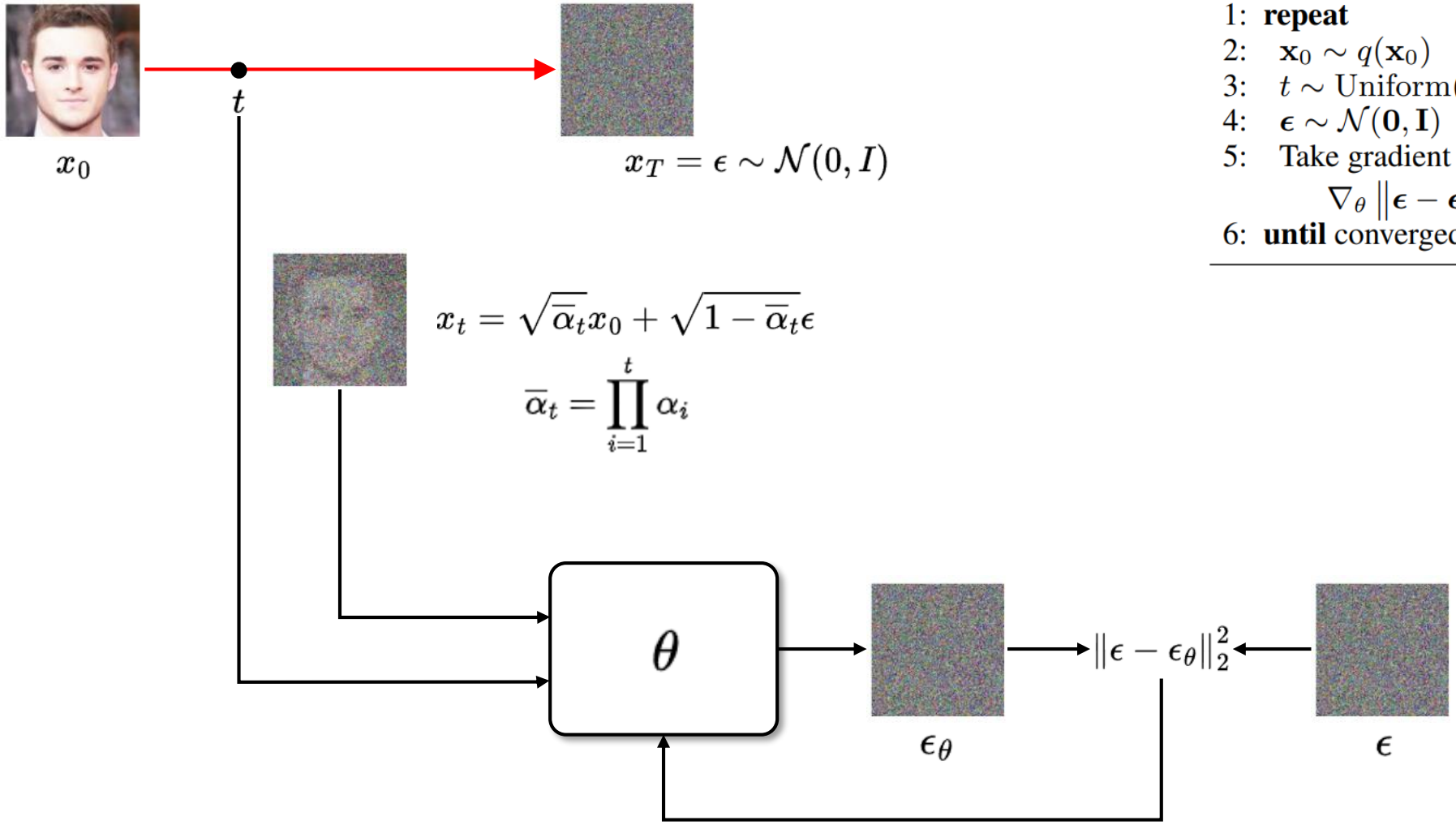
- **Purpose**
  - To explicitly indicate how corrupted the input is at a given timestep
- **Sinusoidal Time Embedding**

$$\mathrm{PE}(t) = \Big[\underbrace{\sin(\omega_0 t),\ \sin(\omega_1 t),\ \ldots,\ \sin(\omega_{\frac{d}{2}-1} t)}_{\text{first half}},\ \underbrace{\cos(\omega_0 t),\ \cos(\omega_1 t),\ \ldots,\ \cos(\omega_{\frac{d}{2}-1} t)}_{\text{second half}}\Big] \qquad \omega_k = 10000^{-\frac{k}{d/2}} \quad \text{for } k = 0, 1, \ldots, \frac{d}{2} - 1$$

$$e_t = \mathrm{MLP}(\mathrm{PE}(t))$$



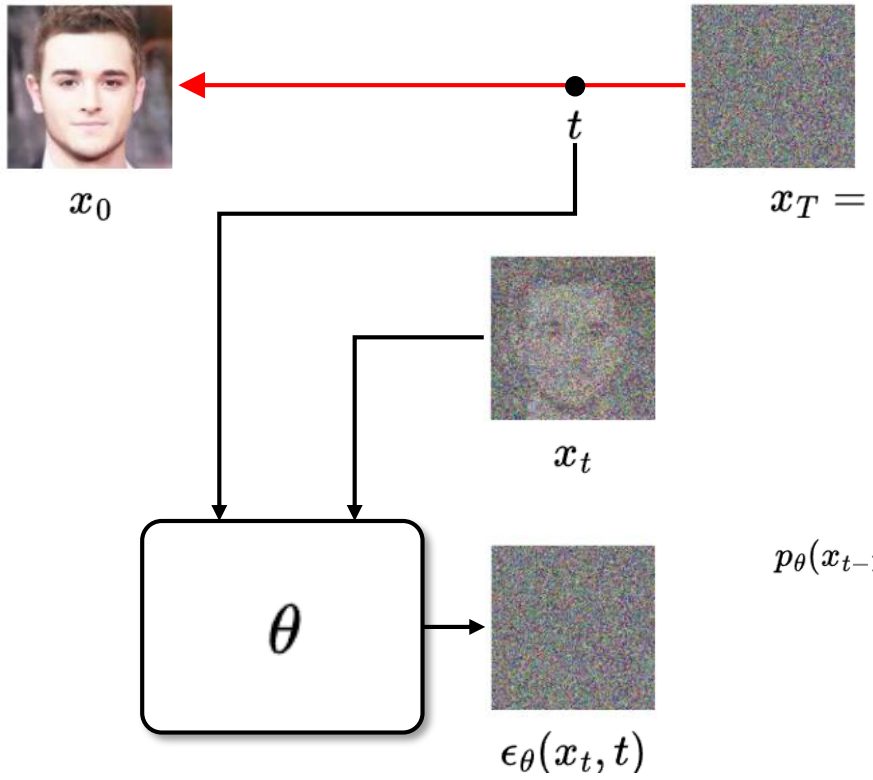
Training



Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Sampling



Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1}; \mu_\theta(t), \sigma_\theta(t)\right)$$

$$= \mathcal{N}\left(x_{t-1}; \left(\frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t} + \frac{1-\alpha_t}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\right)x_t - \frac{(1-\alpha_t)\sqrt{1-\bar{\alpha}_t}}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\epsilon_\theta, \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

$$= \mathcal{N}\left(x_{t-1}; \frac{x_t}{\sqrt{\alpha_t}} - \frac{(1-\alpha_t)\sqrt{1-\bar{\alpha}_t}}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)}\epsilon_\theta, \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right) \quad \Leftarrow \quad \bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$$

$$= \mathcal{N}\left(x_{t-1}; \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta\right), \frac{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathrm{I}\right)$$

$$\mu_\theta(t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right)$$

$$x_{t-1} = \mu_\theta(t) + \sigma_\theta(t)z$$

Experiments

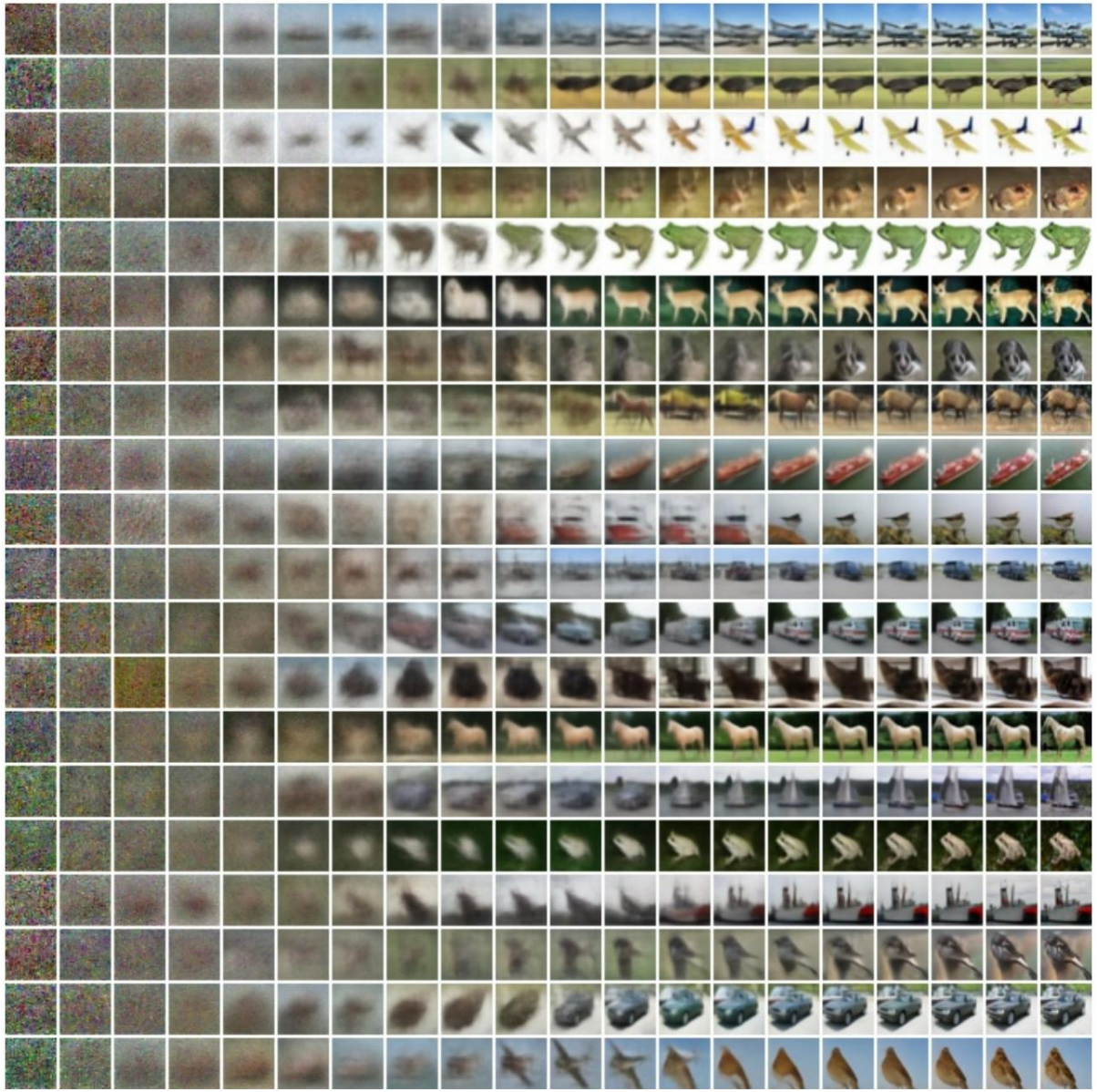


Figure 14: Unconditional CIFAR10 progressive generation

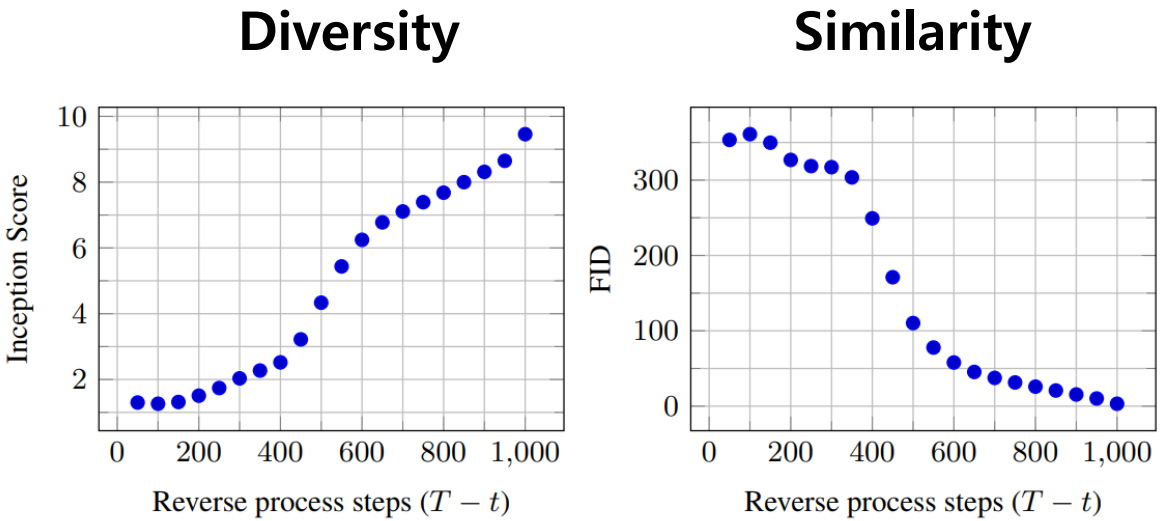


Figure 10: Unconditional CIFAR10 progressive sampling quality over time

Summary

- **Limitations**
  - Sampling is computationally expensive and slow
  - Operation in pixel space leads to high memory and computation costs

- **VAE vs. DDPM**

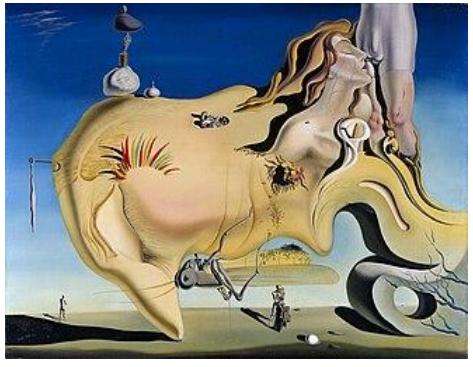
	VAE	**DDPM**
Generative principle	Encoder ⟶ Latent ⟶ Decode	Iterative denoising from Gaussian noise
Training objective	Reconstruction + KL divergence	Noise prediction
Sampling speed	Very fast	Very slow
Sample quality	Often blurry	High, sharp, realistic

Text-to-Image

[DALL·E]

- **Salvador Dalí**
  - Known for highly imaginative, surreal artworks
  - Represents creativity, imagination, and the ability to depict things that do not exist in reality





https://www.smu.edu/news/archives/2016/meadows-dali-acquisition-29jan2016
https://en.wikipedia.org/wiki/Salvador_Dal%C3%AD

The Great Masturbator (1929); oil on canvas, 110 cm × 150 cm, Museo Nacional Centro de Arte Reina Sofía

- **WALL·E – Pixar robot**
  - A small, endearing robot from Pixar's animation
  - Represents technology, automation, and artificial intelligence





https://en.wikipedia.org/wiki/WALL-E
https://en.wikipedia.org/wiki/WALL-E_%28character%29

"DALL·E" combines
Dalí's artistic creativity
with WALL·E's robotic intelligence,
representing an AI system
that creates imaginative images from text !

Overview

- **Motivation**
  - Can we treat images as a "foreign language" and train a single model that jointly understands vision + language?
    → Generative reasoning using natural language
  - Enabling compositional image generation from text, even for combinations not seen during training
- **Goal**
  - Mapping text prompts to images in a zero-shot setting



(a) a tapir made of accordion. a tapir with the texture of an accordion.

(b) an illustration of a baby hedgehog in a christmas sweater walking a dog

(c) a neon sign that reads "backprop". a neon sign that reads "backprop". backprop neon sign

(d) the exact same cat on the top as a sketch on the bottom

Figure 2. With varying degrees of reliability, our model appears to be able to combine distinct concepts in plausible ways, create anthropomorphized versions of animals, render text, and perform some types of image-to-image translation.

Background

- **VQ-VAE** (dVAE in the paper)
  - Converts images to discrete tokens (latent codes)
  - Enables training of transformers on image sequences

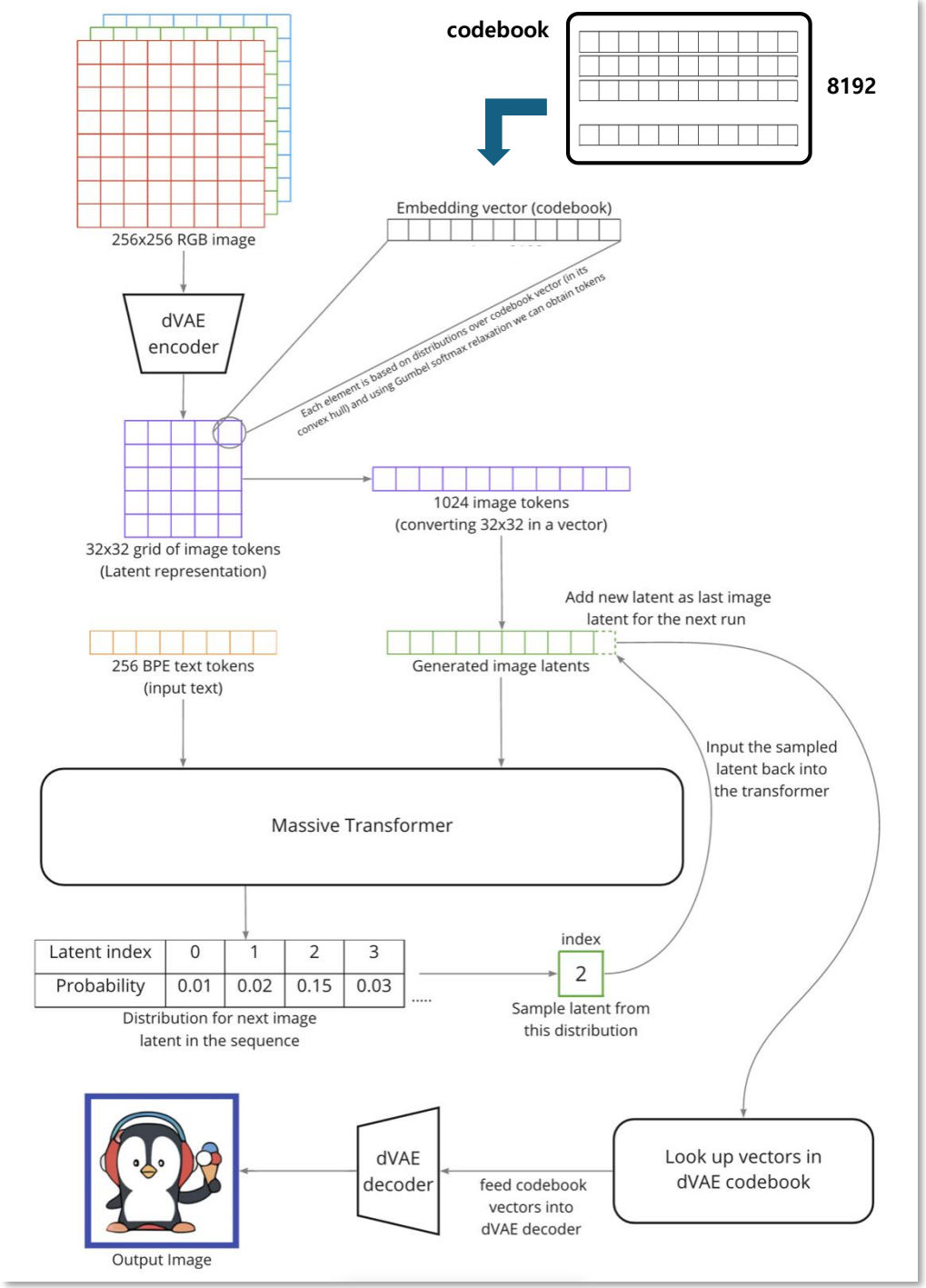
- **GPT** architecture
  - Autoregressive sequence model predicting next token

- **CLIP**
  - Understanding natural language and images jointly
  - Scores images and selects top-k results during inference → Semantic judge

Architecture

- dVAE **encoder**
  - compresses images into discrete tokens
- **Transformer**
  - generates these tokens autoregressively conditioned on text
- dVAE **decoder**
  - reconstructs the final image from the generated latent tokens

- **Large scale up** for training
  - Parameters : 12B
  - Text-image pair dataset : 250M



https://jashrathod.github.io/2021-12-13-zero-shot-text-to-image-generation/

Loss

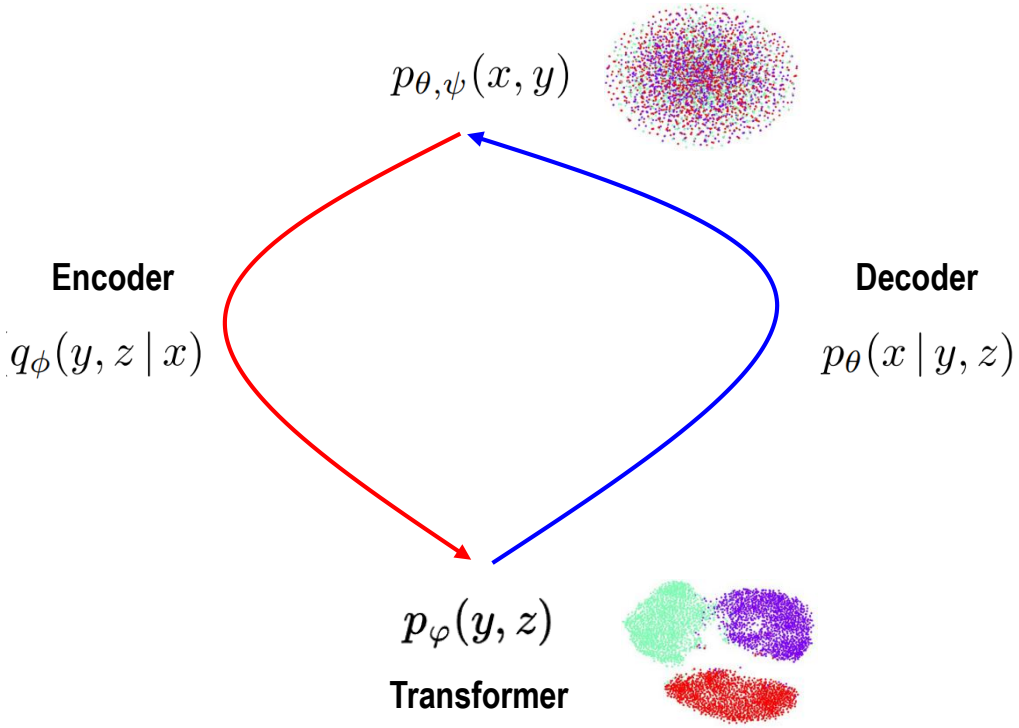
- **Maximizing the ELBO (Evidence Lower Bound)** of the joint likelihood

Reconstruction **Regularization**

$$\ln p_{\theta,\psi}(x,y) \geqslant \mathop{\mathbb{E}}_{z\sim q_\phi(z\,|\,x)} \left(\ln p_\theta(x\,|\,y,z) - \beta\, D_{\mathrm{KL}}(q_\phi(y,z\,|\,x), p_\psi(y,z)) \right)$$

images x
captions y
tokens z

- q_ϕ denotes the distribution over the 32×32 image tokens generated by the dVAE encoder given the RGB image x ;
- p_θ denotes the distribution over the RGB images generated by the dVAE decoder given the image tokens; and
- p_ψ denotes the joint distribution over the text and image tokens modeled by the transformer.

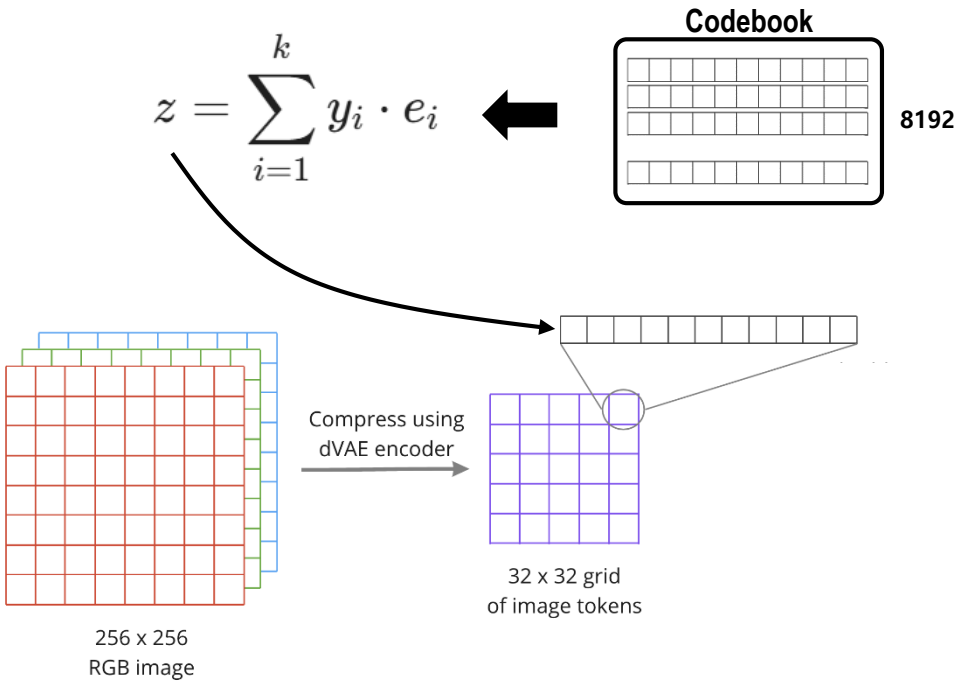


Train (1/3)

- **Stage 1 (Representation learning)**
  - Training the encoder, codebook, and decoder for image reconstruction
    - Learns a discrete VAE that maps 256x256 RGB images to a 32x32 grid of latent tokens
  - Learning the visual codebook
    - Replacing 'argmax' with 'Gumbel softmax relaxation' with annealing
      → Back-propagation

$$y_i = \frac{\exp((\log(\pi_i) + g_i)/\tau)}{\sum_{j=1}^{k} \exp((\log(\pi_j) + g_j)/\tau)}$$

$$z = \sum_{i=1}^{k} y_i \cdot e_i$$



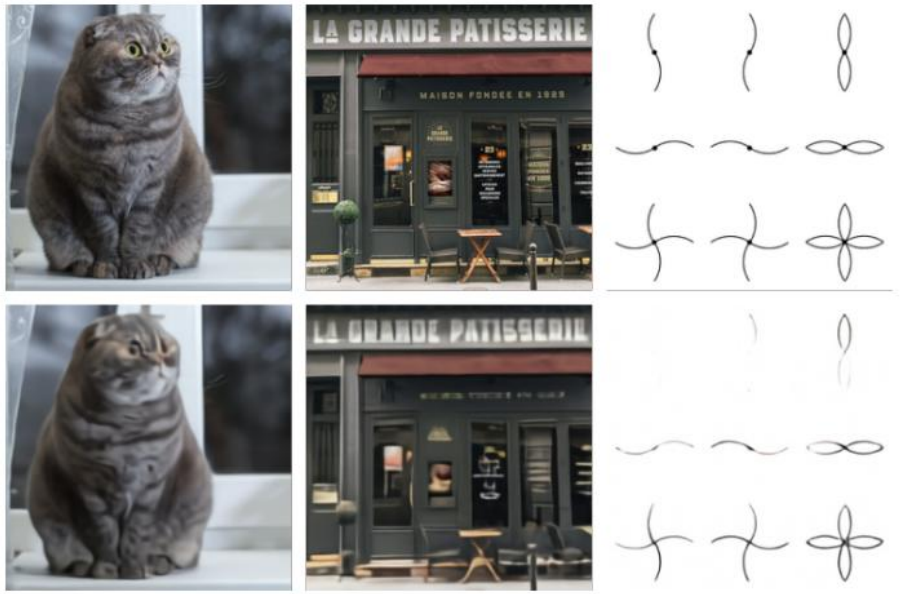
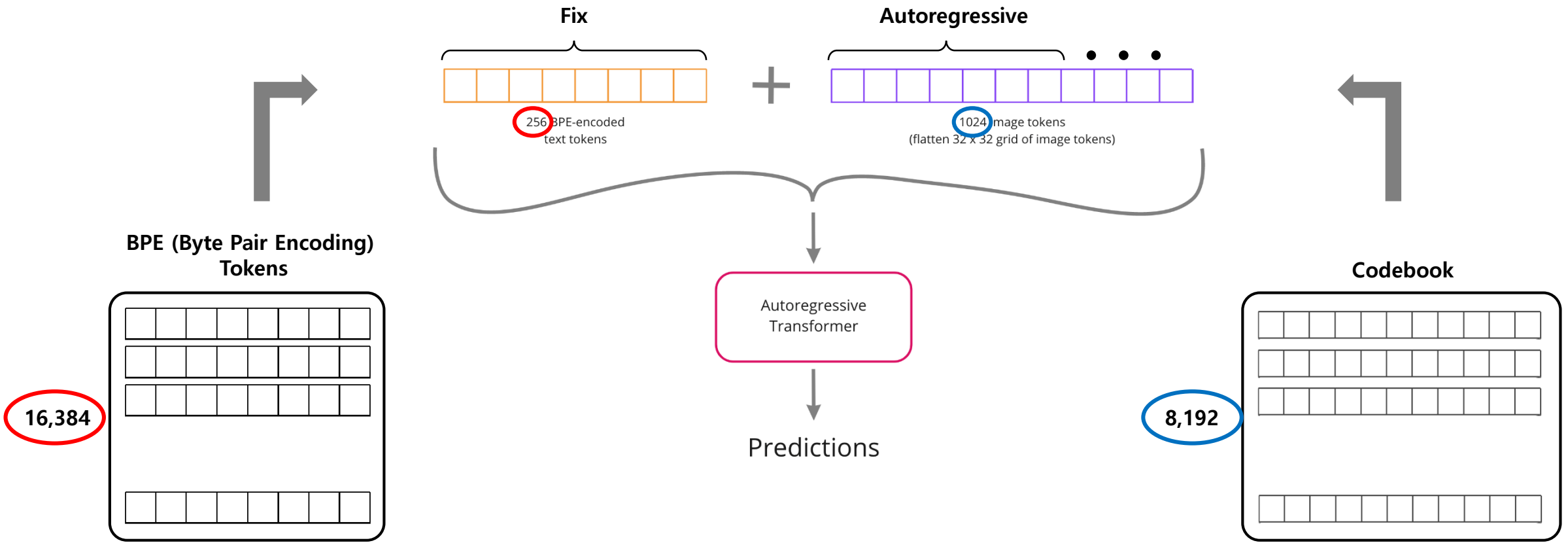


Figure 1. Comparison of original images (top) and reconstructions from the discrete VAE (bottom). The encoder downsamples the spatial resolution by a factor of 8. While details (e.g., the texture of the cat's fur, the writing on the storefront, and the thin lines in the illustration) are sometimes lost or distorted, the main features of the image are still typically recognizable. We use a large vocabulary size of 8192 to mitigate the loss of information.

A bit blurry but recognizable !

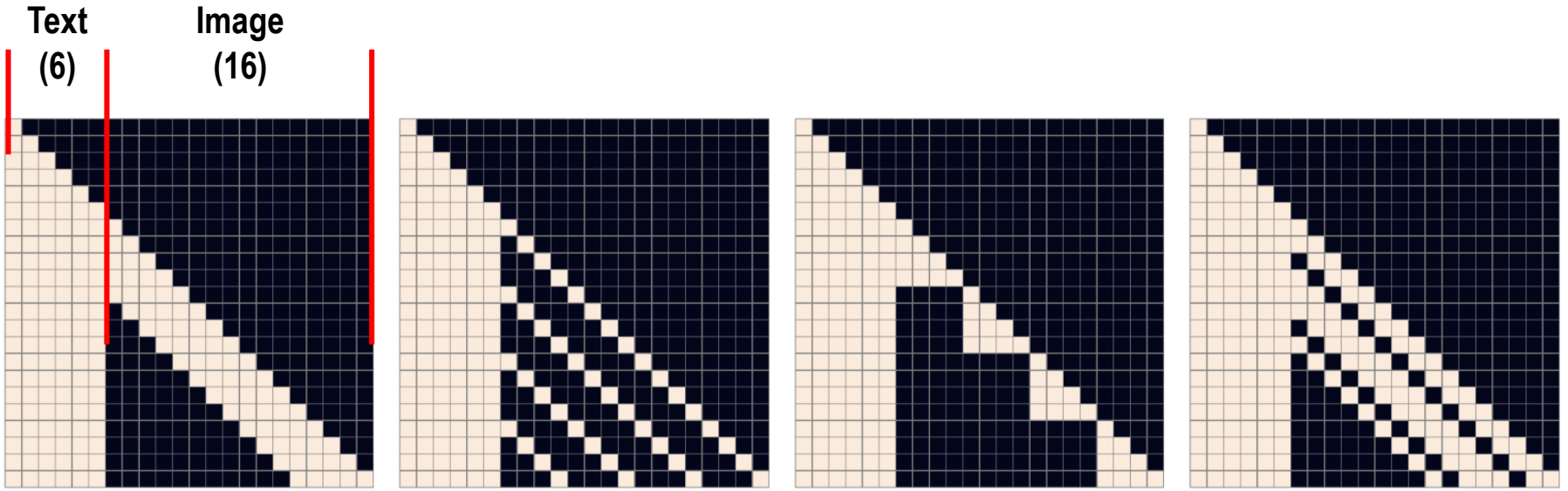
Train (2/3)

- **Stage 2 – 1 (Prior learning)**
  - Training a large autoregressive transformer on text tokens + image latent tokens
    - Predicting the next token, i.e., generating image latents conditioned on text
    - Acts as the prior, replacing the PixelCNN-style prior used in traditional VQ-VAE setups.



Train (3/3)

- Stage 2 - 2
  - **Applying causal attention mask in transformer**

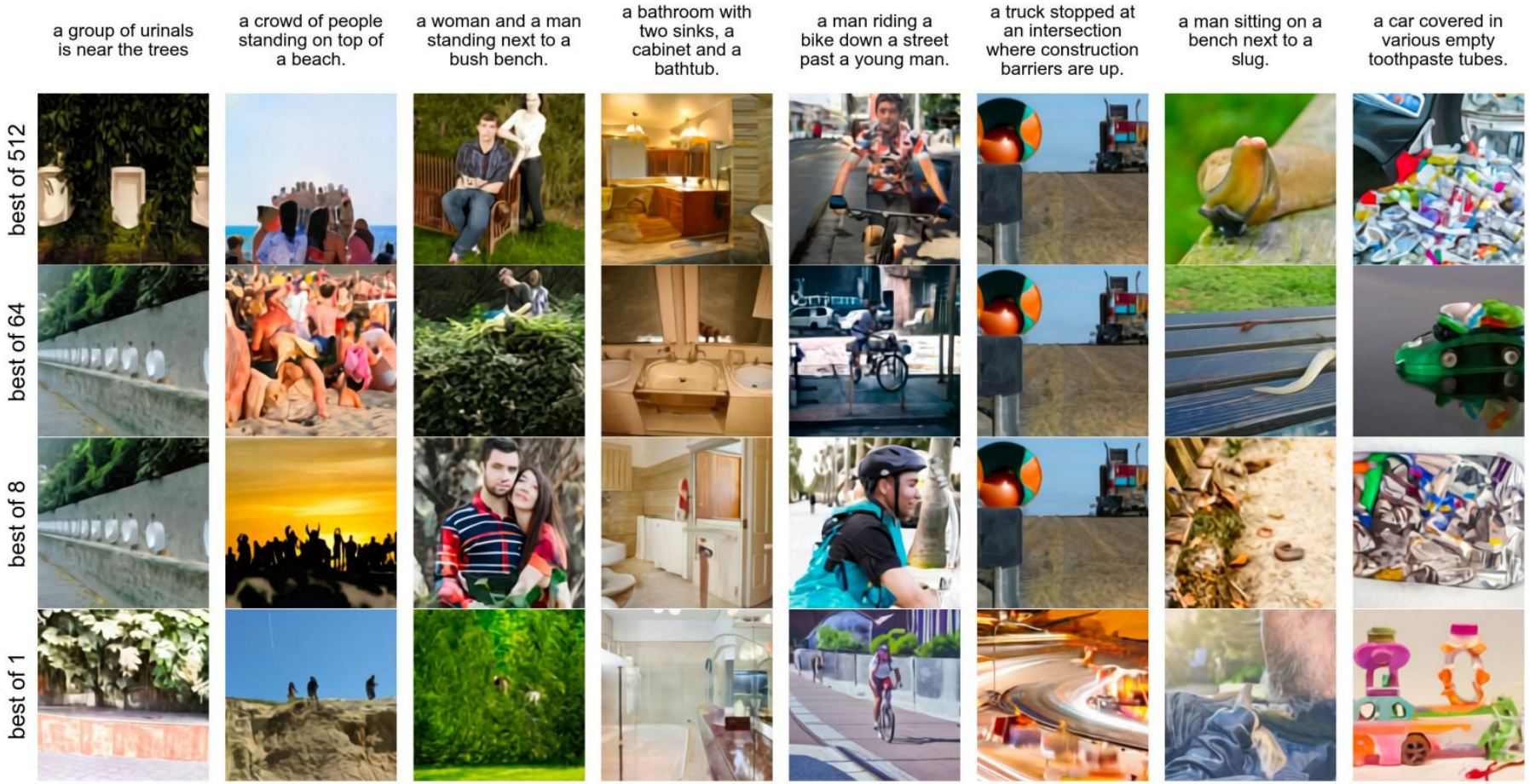


(a) Row attention mask. (b) Column attention mask. (c) Column attention mask with transposed image states. (d) Convolutional attention mask.

Figure 11. Illustration of the three types of attention masks for a hypothetical version of our transformer with a maximum text length of 6 tokens and image length of 16 tokens (i.e., corresponding to a 4×4 grid). Mask (a) corresponds to row attention in which each image token attends to the previous 5 image tokens in raster order. The extent is chosen to be 5, so that the last token being attended to is the one in the same column of the previous row. To obtain better GPU utilization, we transpose the row and column dimensions of the image states when applying column attention, so that we can use mask (c) instead of mask (b). Mask (d) corresponds to a causal convolutional attention pattern with wraparound behavior (similar to the row attention) and a 3×3 kernel. Our model uses a mask corresponding to an 11×11 kernel.

Inference

- **Generate** M candidate latent sequences ➡ $z^{(i)} = (z_1^{(i)}, z_2^{(i)}, \ldots, z_N^{(i)}), \quad i = 1, 2, \ldots, M$
- **Decode** latent tokens into images ➡ $I_i = \text{Decoder}(z^{(i)}) \quad \text{for } i = 1, \ldots, M$
- **Score** each image using CLIP ➡ $\text{score}(I_i) = \cos\left(f_{\text{text}}(\text{prompt}),\ f_{\text{image}}(I_i)\right)$



Experiments (1/3)

- Comparison with **other approaches**

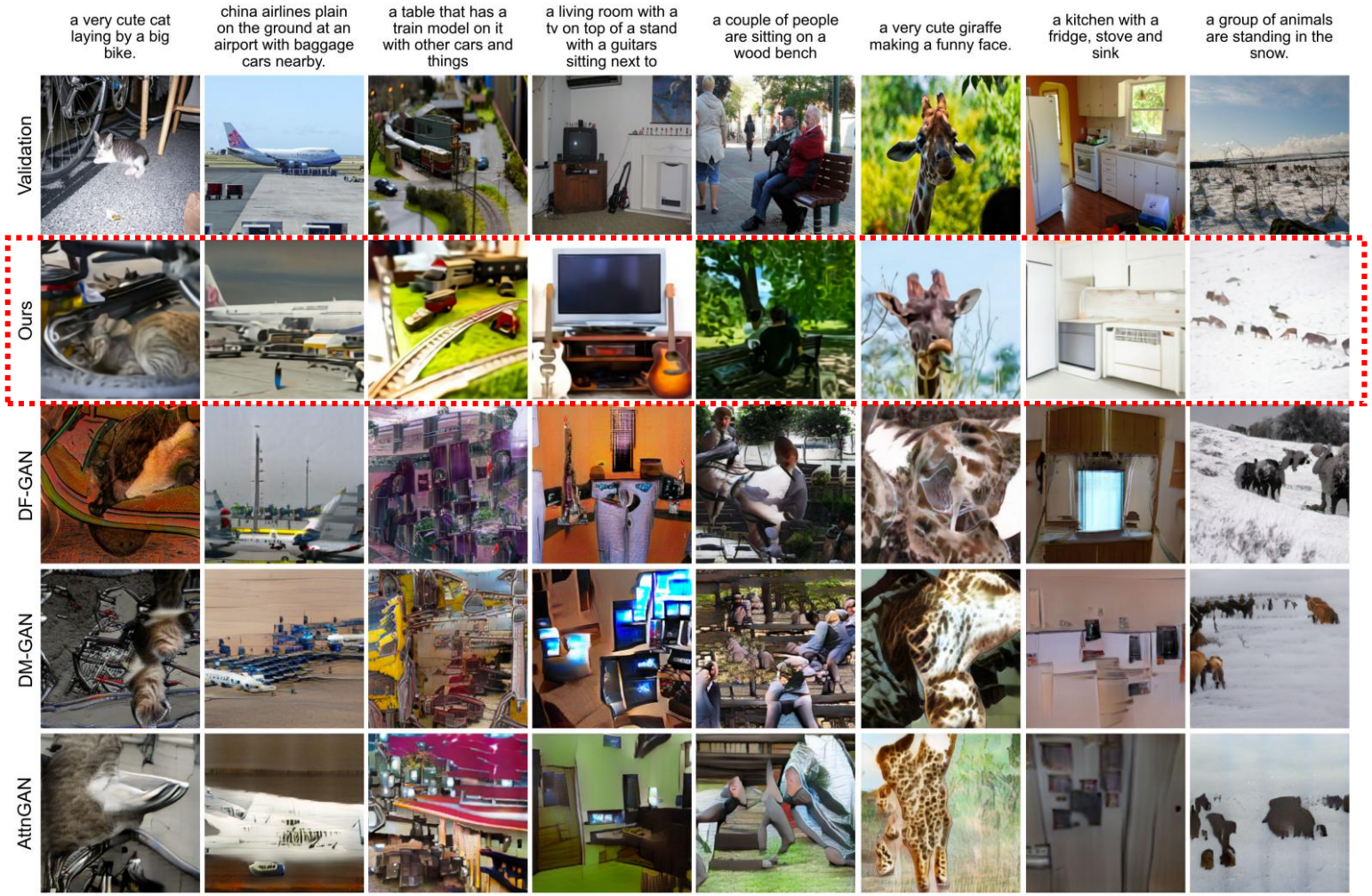
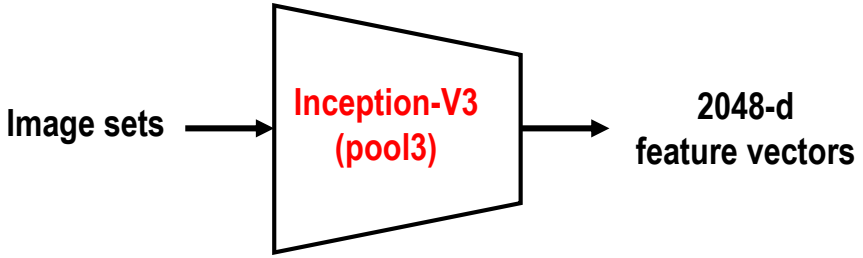


Figure 3. Comparison of samples from our model to those from prior approaches on captions from MS-COCO. Each of our model samples is the best of 512 as ranked by the contrastive model. We do not use any manual cherrypicking with the selection of either the captions or the samples from any of the models.

Experiments (2/3)

- Comparison in terms of **FID (Fréchet Inception Distance)**
  - Metric that measures how close the distribution of generated images is to the one of real images in a deep feature space

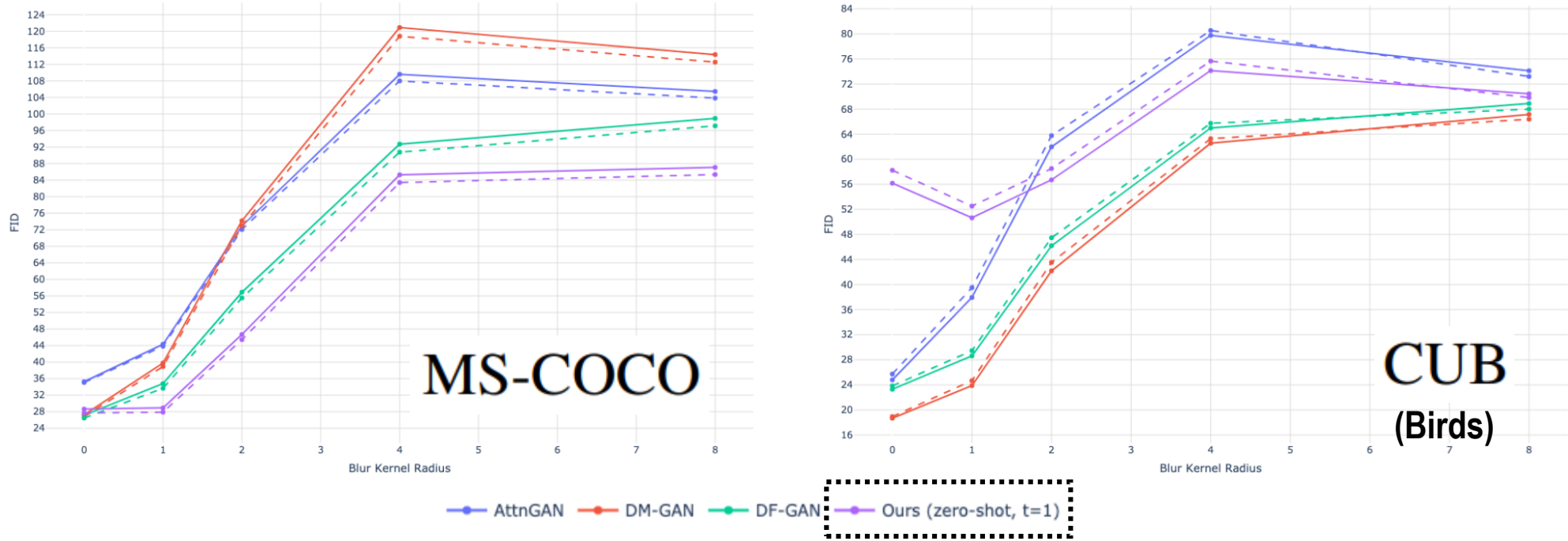


Real image feature distribution $\mathcal{N}(\mu_r, \Sigma_r)$

Generated image feature distribution $\mathcal{N}(\mu_g, \Sigma_g)$

$$FID = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

(Lower is better !)



Experiments (3/3)

- **Image-to-image translation** → Zero-shot
  - Partial image conditioning and autoregressive completion



(a) "the exact same cat on the top as a sketch on the bottom"



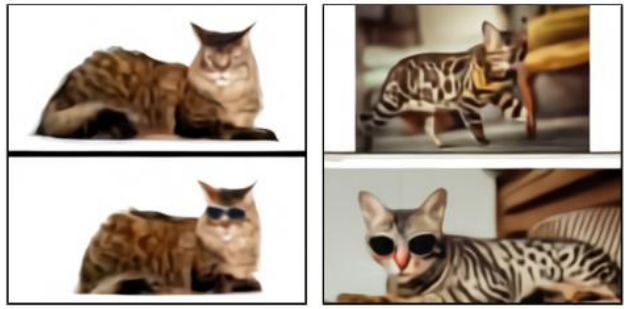
(b) "the exact same photo on the top reflected upside-down on the bottom"



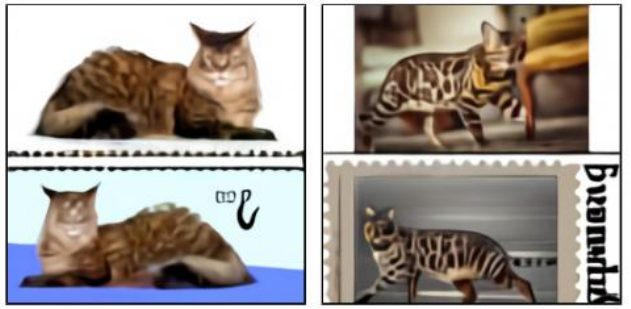
(c) "2 panel image of the exact same cat. on the top, a photo of the cat. on the bottom, an extreme close-up view of the cat in the photo."



(d) "the exact same cat on the top colored red on the bottom"



(e) "2 panel image of the exact same cat. on the top, a photo of the cat. on the bottom, the cat with sunglasses."



(f) "the exact same cat on the top as a postage stamp on the bottom"

[Stable diffusion]

High-Resolution Image Synthesis with Latent Diffusion Models, 2022

- A **Latent Diffusion Model (LDM)**
  - Generates high-fidelity images by performing the diffusion process within a compressed Latent Space
  - Uses Conditioning (commonly text) to steer the denoising process

Prompt : Cleopatra as a spaceship commander



https://simplified.com/ai-design/ai-image-generator/cleopatra-as-a-spaceship-comma-3bbf4c37-037d-4b9f-a1d4-f890356f2914

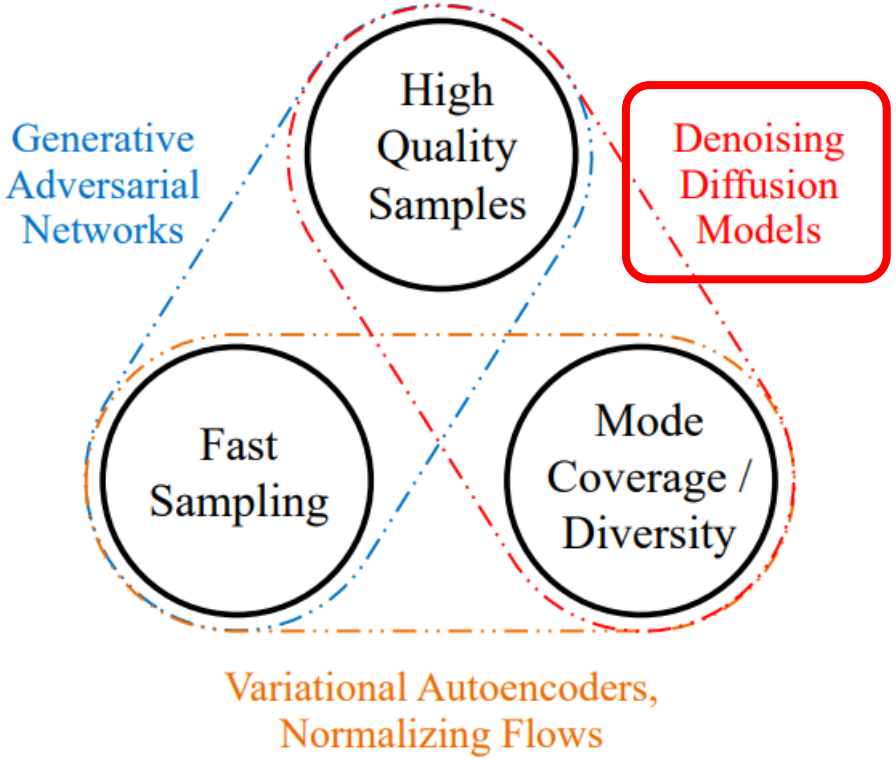
Prompt : A master jedi cat in star wars, wearing a jedi cloak hood



https://www.reddit.com/r/StableDiffusion/comments/14jx0dj/a_master_jedi_cat_in_star_wars_wearing_a_jedi/

Diffusion models

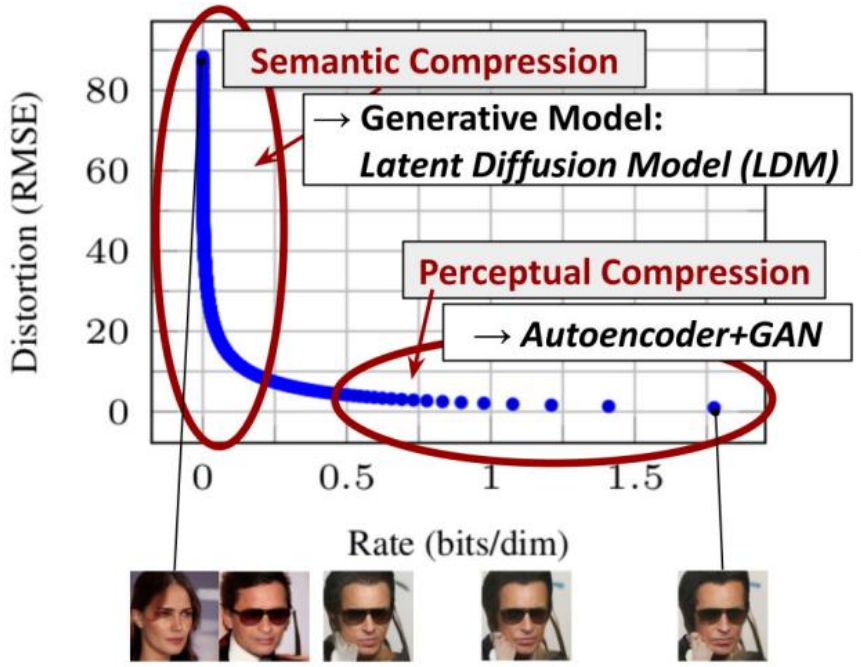
- Optimize sample quality and training stability
- But, **sacrifice sampling speed** because of iterative refinement process in **pixel space**
- Only **unconditional models**



Generative Model Trilemma

Overview (1/2)

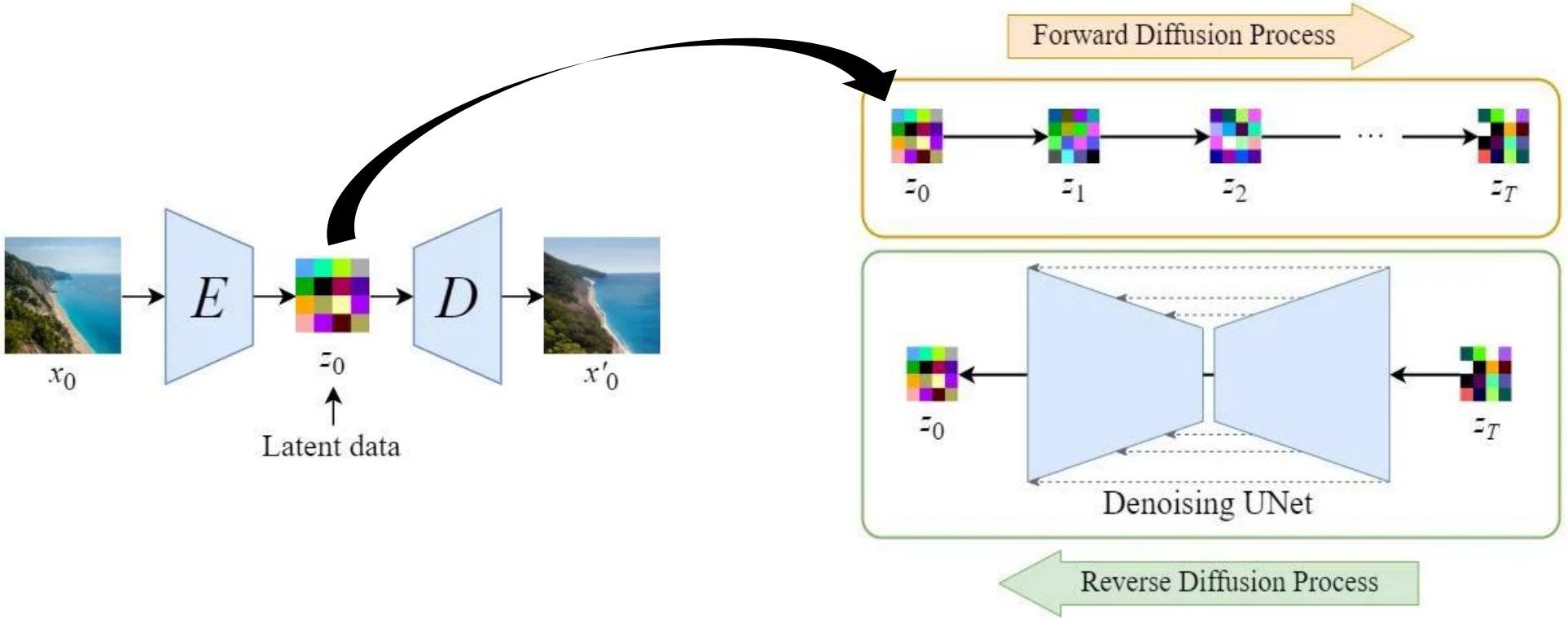
- **Motivation**
  - Pixel space bottleneck
    - Traditional DMs operate in the high dimensional pixel space → Massive amounts of VRAM and GPU hours
  - Redundancy in visual data
    - High-frequency details are "perceptual redundancy" → Semantic noise





Overview (2/2)

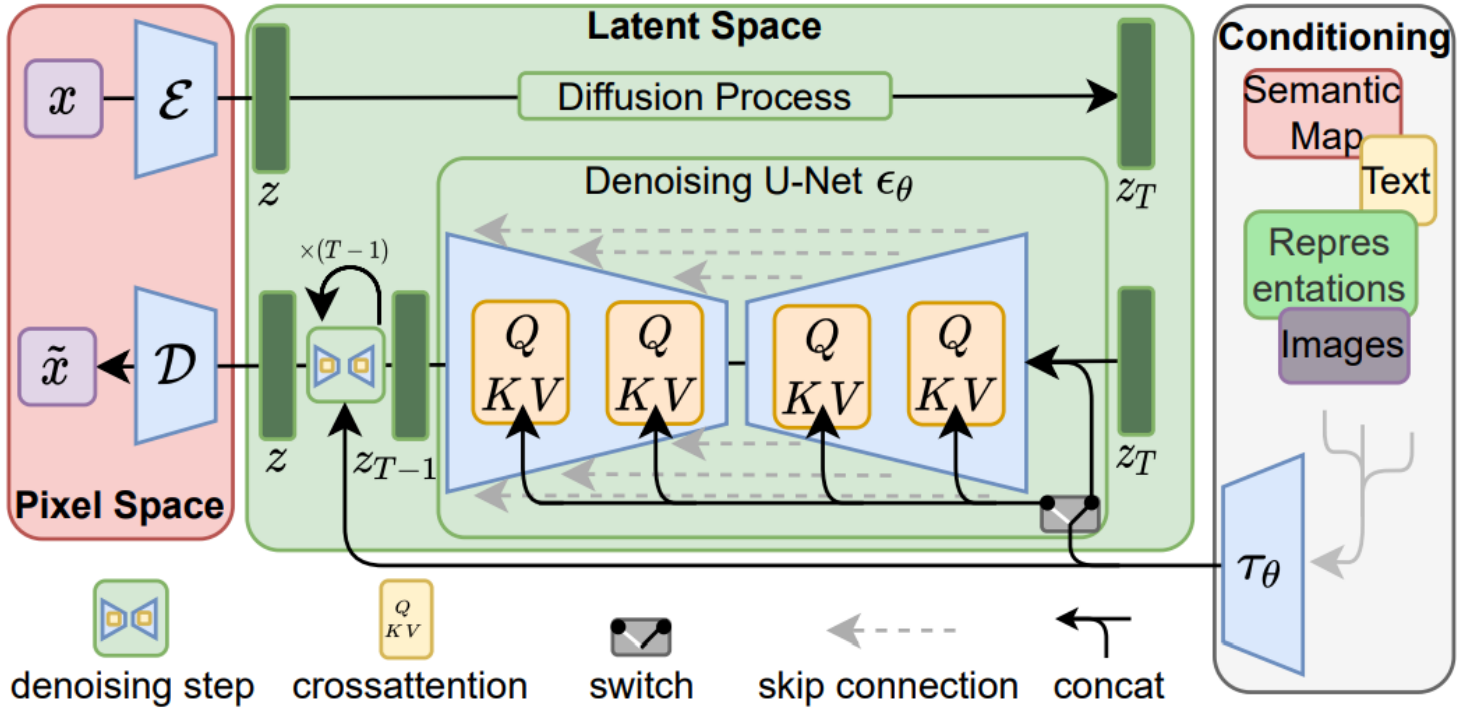
- **Goal**
  - Decoupling the generation process from the high-dimensional pixel space without sacrificing visual fidelity
    - Diffusion models in a compressed (but semantically equivalent) latent space
    - Universal conditioning mechanism based on text, bounding box, segmentation map, etc.



https://codoraven.com/blog/ai/stable-diffusion-clearly-explained/

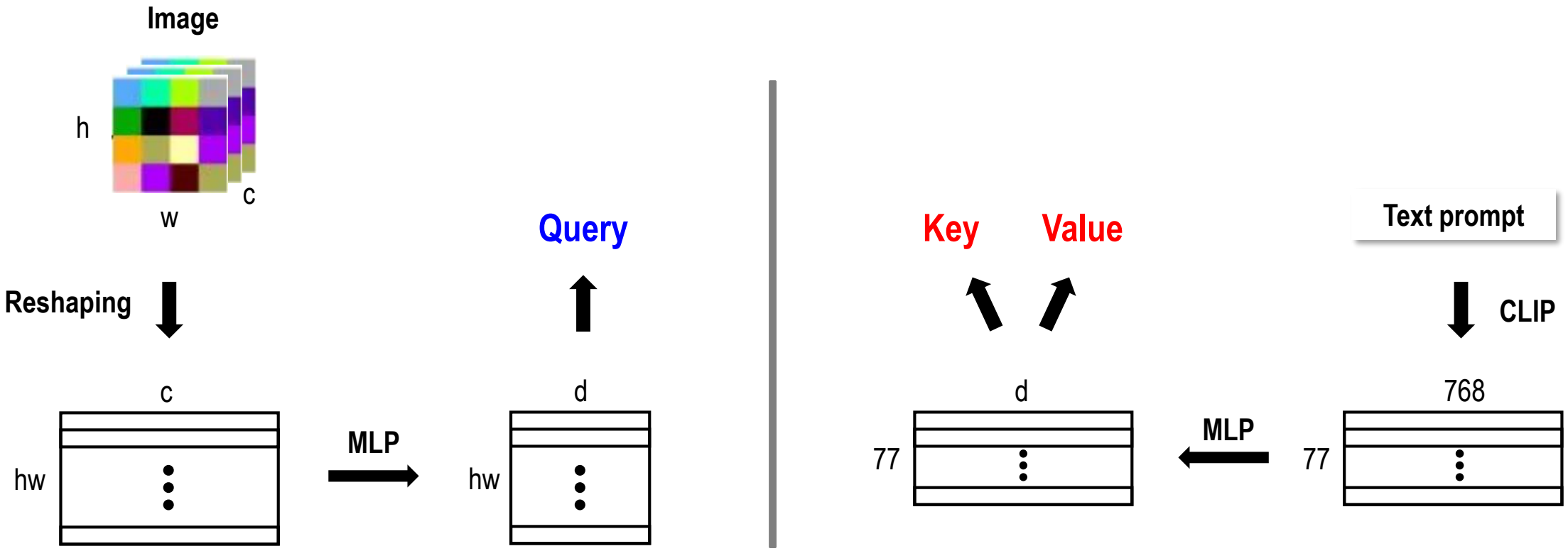
Architecture

- **VAE**
  - Encoder ($\mathcal{E}$) – Compressing the image into a low-dimensional latent representation z
  - Decoder ($\mathcal{D}$) – Reconstructing the latent representation z back into a high-resolution pixel image
- **U-Net noise predictor**
  - Predicts noise within the latent space
- **Conditioning**
  - Injects external information (text, semantic map, etc.) into the U-Net



Conditioning

- **For text inputs**,
  - First, convert them into embeddings (vectors) using a language model (e.g. BERT, CLIP)
  - Then, they are mapped into the U-Net via the (multi-head) Attention(Q,K,V) layer
- **For other spatially aligned inputs** (e.g. semantic maps, images, inpainting),
  - Concatenation with U-Net inputs



Training (1/2)

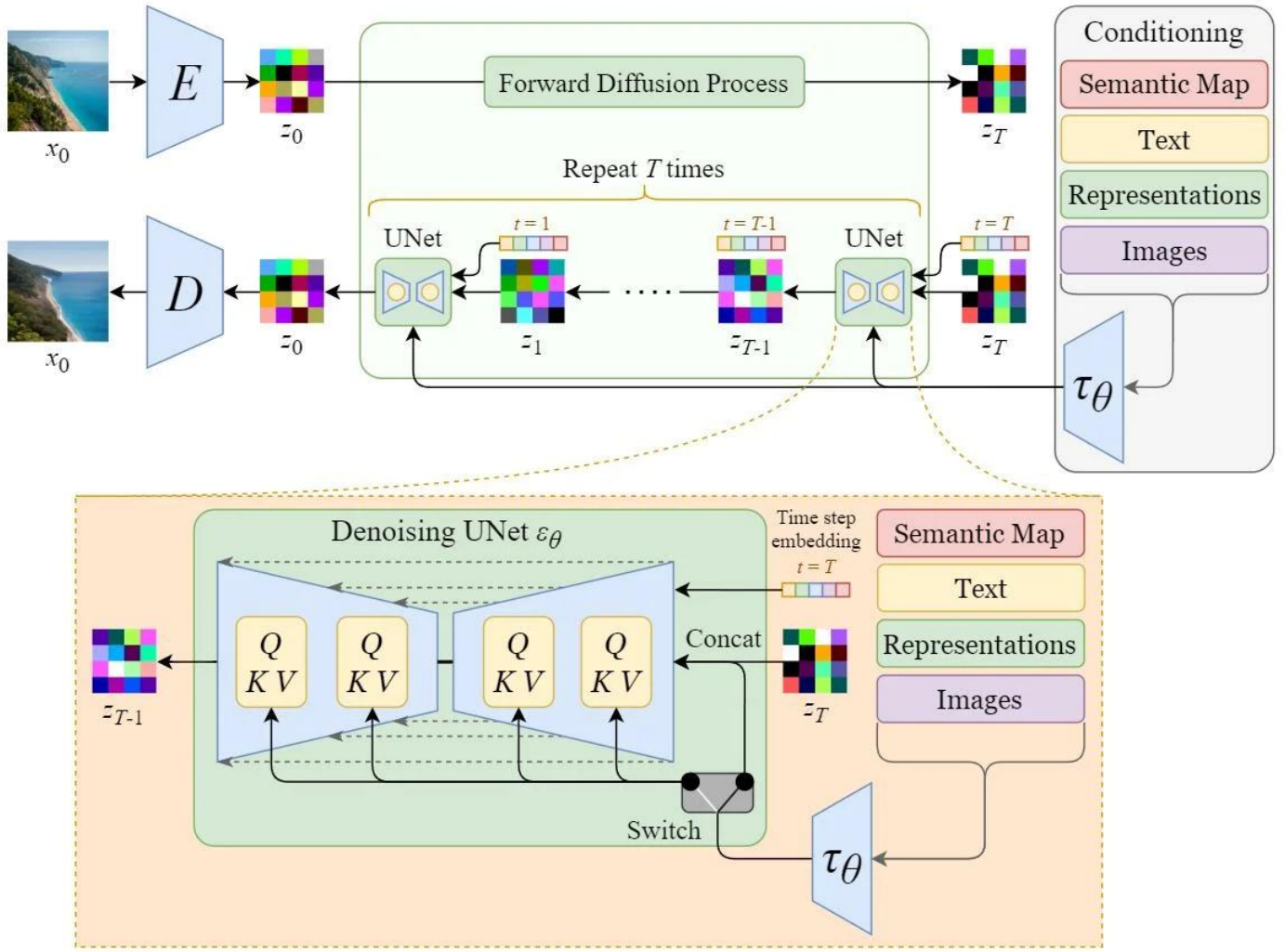
- **LDM (Latent Diffusion Model) Loss**
  - Input latent data z_t instead of the image x_t
  - Adding conditioning input $\tau_\theta(y)$ to the U-Net

$$z_0 = E(x_0)$$

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$$

$$L_{\text{LDM}} = \mathbb{E}_{t, z_0, \varepsilon, y}\left[\|\varepsilon - \varepsilon_\theta(z_t, t, \tau_\theta(y))\|^2\right]$$

Conditioning



https://codoraven.com/blog/ai/stable-diffusion-clearly-explained/

Training (2/2)

- **Dataset**
  - LAION-5B (filtered subset)
  - About 2~3 billion image-text pairs
    - Web-scale image-caption pairs
    - No manual annotations
- **GPU**
  - NVIDIA A100 (40GB/ 80GB VRAM) * 256~1024 GPUs
  - Mixed precision (FP16)
- **Total parameters**
  - 1.06B
    - U-Net 860M + VAE 83M + Text Encoder 123M

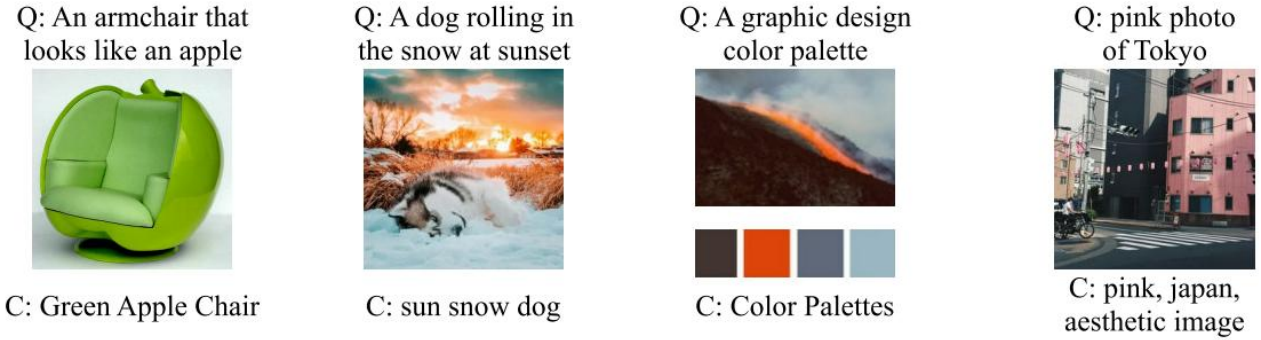
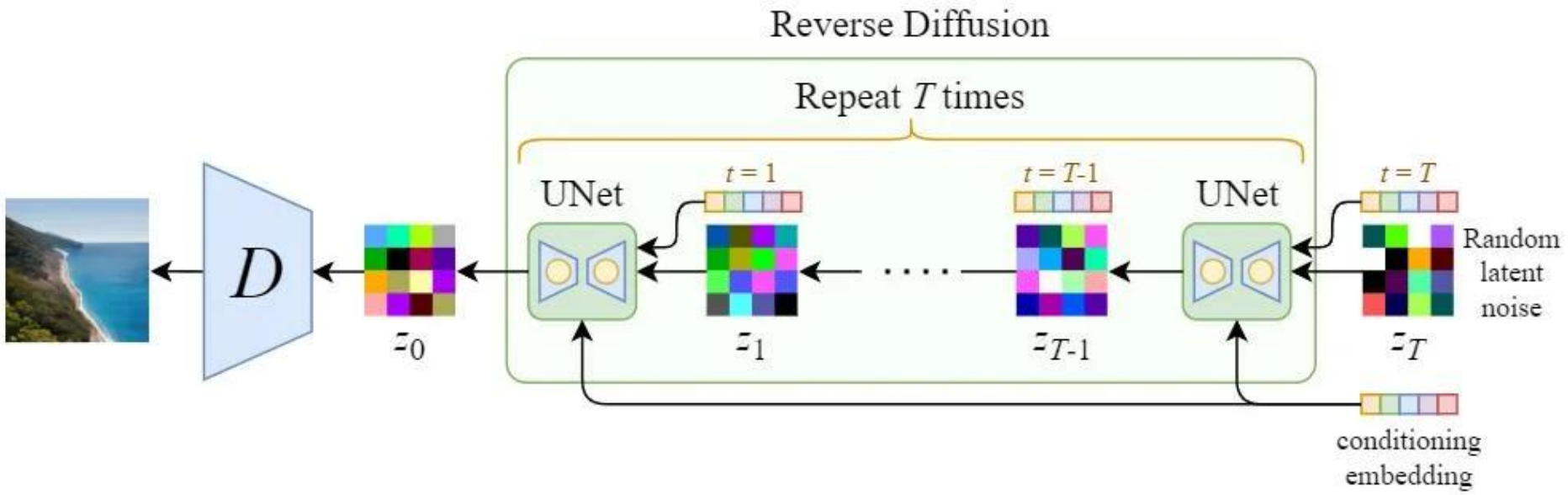


Figure 3: **LAION-5B examples.** Sample images from a nearest neighbor search in LAION-5B using CLIP embeddings. The image and caption (C) are the first results for the query (Q).

Sampling

- Sampling **starts from random noise in the latent space**
- A diffusion model **iteratively removes noise, guided by the text embedding** through cross-attention
- The final denoised latent is decoded by a **VAE to produce the image**



https://codoraven.com/blog/ai/stable-diffusion-clearly-explained/

Experiments

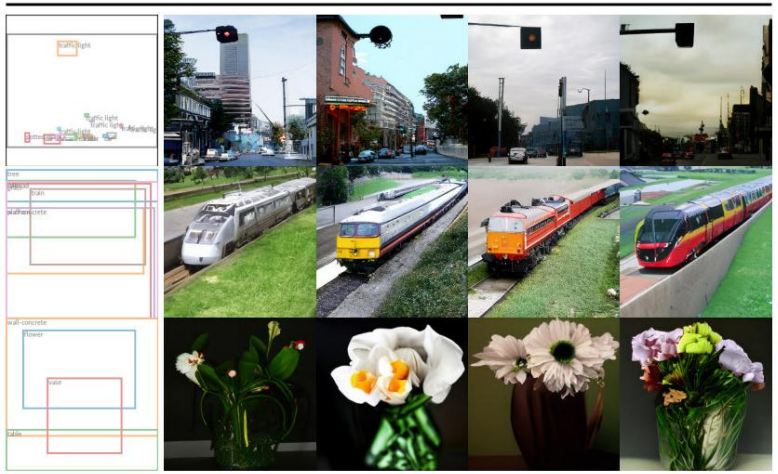


Figure 8. Layout-to-image synthesis with an *LDM* on COCO [4], see Sec. 4.3.1. Quantitative evaluation in the supplement D.3.



Figure 10. ImageNet 64→256 super-resolution on ImageNet-Val. *LDM-SR* has advantages at rendering realistic textures but SR3 can synthesize more coherent fine structures. See appendix for additional samples and cropouts. SR3 results from [72].



Figure 9. A *LDM* trained on 256^2 resolution can generalize to larger resolution (here: 512×1024) for spatially conditioned tasks such as semantic synthesis of landscape images. See Sec. 4.3.2.

Summary

- **Contributions**
  - Introduces a latent diffusion framework that performs diffusion in a compressed latent space instead of pixel space
  - Enables high-quality text-to-image generation with much higher efficiency than pixel-based diffusion models
  - Makes large-scale diffusion models practical and accessible for both research and real-world applications





Course review

